

데이터 링크 제어(DLC)

목 차

- DLC 서비스
- 데이터 링크층 프로토콜
- HDLC
- 점-대-점 프로토콜

DLC 서비스

■ 프레임 짜기

- ▶ 비트들을 프레임으로 만들고 각 프레임은 다른 프레임과 구분
- ▶ 예: 우편 시스템
 - 봉투에 편지를 넣는 간단한 행위로 인해 봉투가 분리 기능을 함으로써 서로 다른 정보가 뒤섞이지 않도록 하는 것

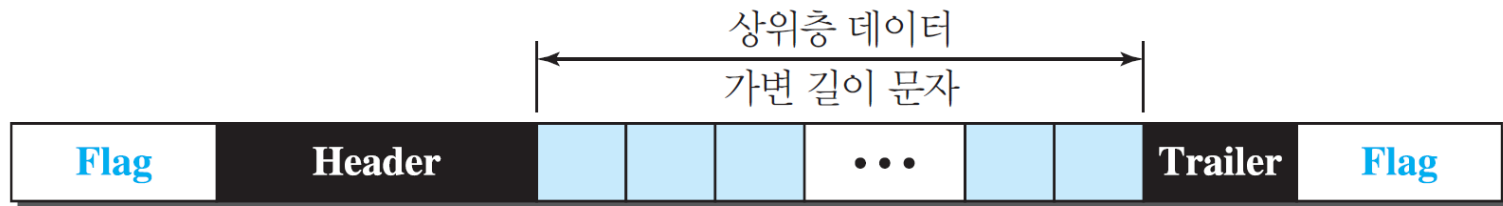
DLC 서비스

- ▶ 프레임 길이
 - 고정 길이 프레임
 - ✓ 고정 길이
 - ✓ 예: ATM 광역네트워크의 셀(cell)
 - 가변 길이 프레임
 - ✓ 주로 LAN에서 사용
 - ✓ 프레임이 끝나는 곳과 다음 프레임이 시작하는 곳 지정
 - » 문자 중심 프레임 짜기
 - » 비트 중심 프레임 짜기

프레임 짜기

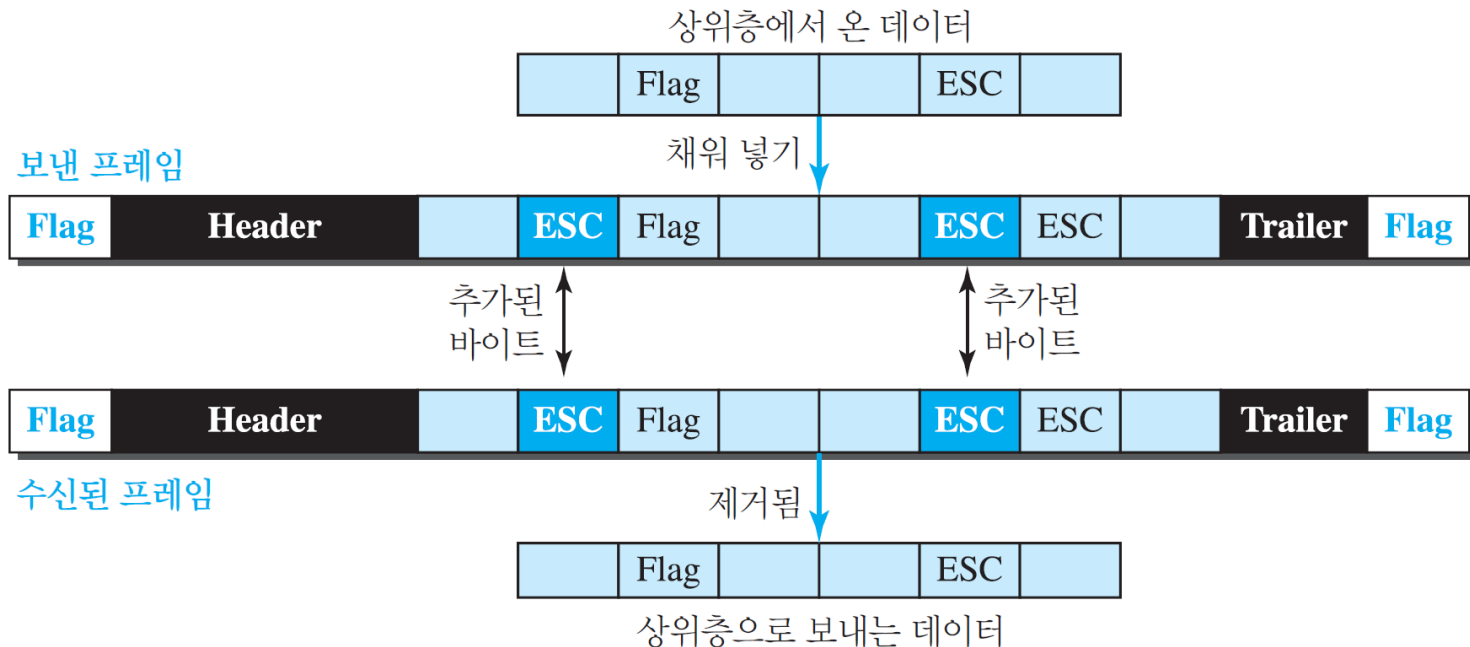
▶ 문자 중심 프레임 짜기

- 전달되는 데이터는 ASCII와 같은 부호화 시스템의 8비트 문자
- 시작과 마지막에 8비트 플래그(flag)를 추가



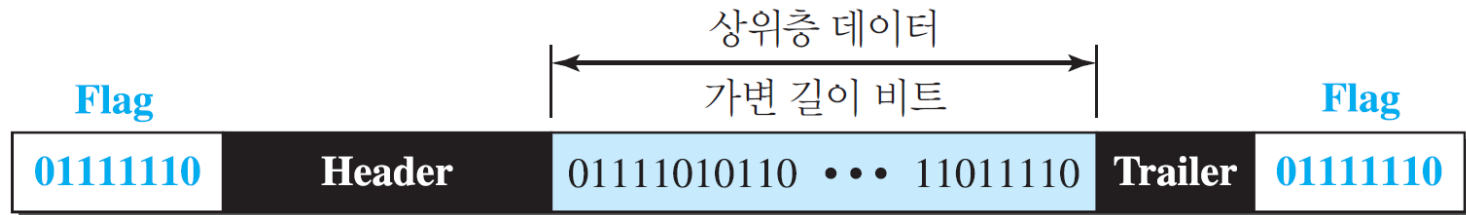
프레임 짜기

- 바이트 채워기(stuffing)와 빼기(unstuffing)
 - ✓ 정보 내에 플래그로 사용되는 패턴이 있을 경우 이를 플래그로 오인하지 않게 하기 위한 것
 - ✓ 바이트 채워기는 텍스트에 플래그나 ESC 문제가 있을 때 여분의 1 바이트를 추가



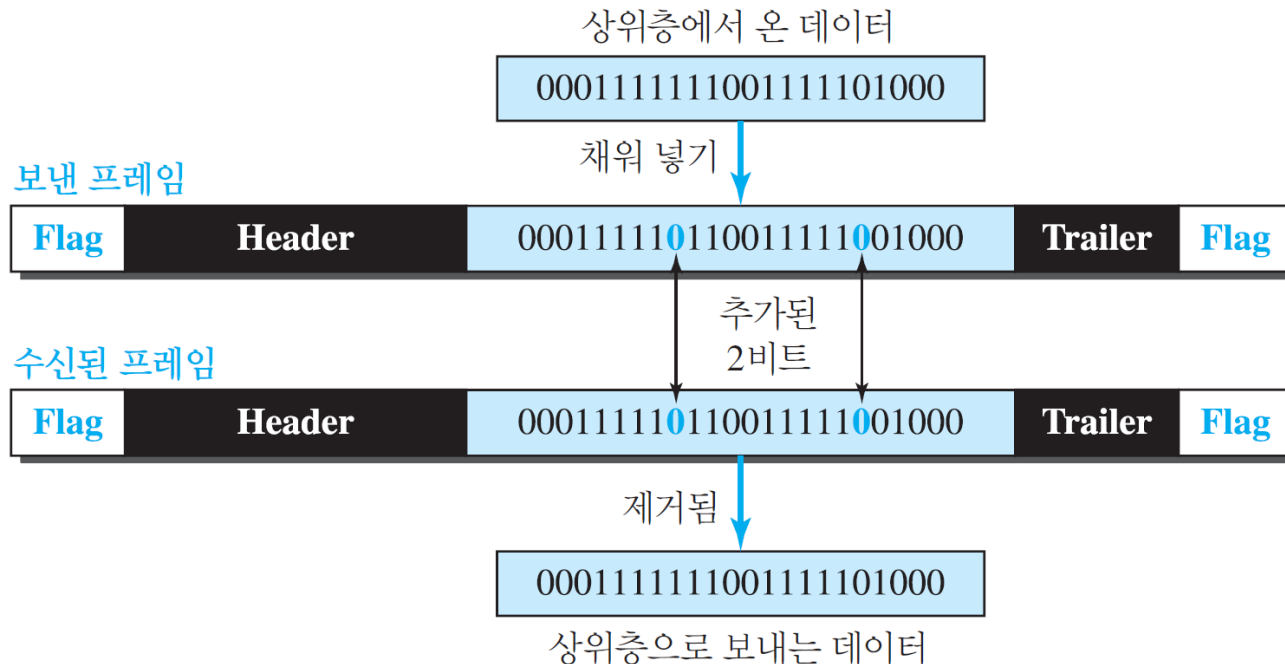
프레임 짜기

- ▶ 비트 중심 프레임 짜기
 - 프레임의 데이터 부분을 전부 bit 열로 인식
 - '01111110' 8비트 패턴의 플래그 사용



프레임 짜기

- ▶ 비트 채우기(stuffing)와 빼기(unstuffing)
 - 수신자는 데이터 속의 '01111110'을 플래그로 오인하지 않도록 하는 것
 - 0 뒤에 연속하는 5개의 1이 있게 되면 0을 추가로 채우는 과정



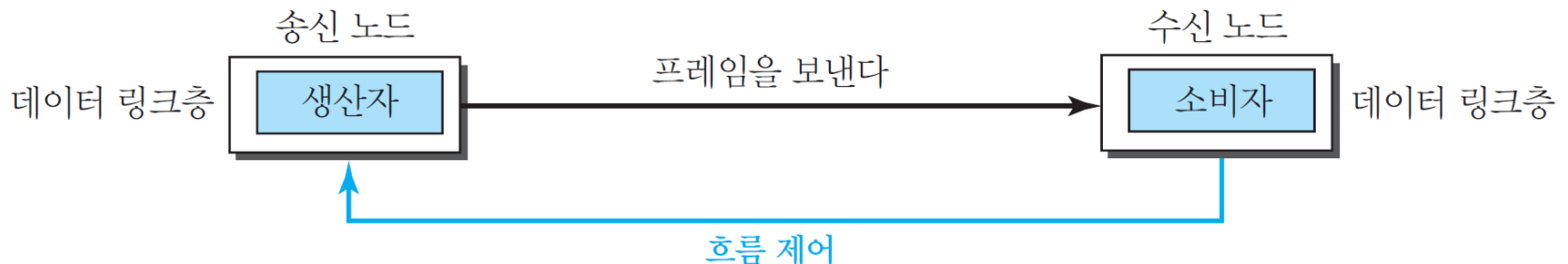
■ 흐름 제어와 오류 제어

▶ 흐름 제어(flow control)

- 수신자로부터 확인응답을 받기 전까지 얼마나 많은 데이터를 전송할 수 있는지를 알려주는 일련의 절차
- 2개의 버퍼(buffer) 사용
 - ✓ 패킷을 저장할 수 있는 메모리의 집합
 - ✓ 하나는 송신 데이터 링크층에서 사용
 - ✓ 다른 하나는 수신 데이터 링크층에서 사용

▶ 오류 제어(error control)

- 손상된 패킷을 수신 노드로 전달 하는 것을 방지하기 위해
- 자동 반복 요구(ARQ)
 - ✓ 오류가 확인된 순간 해당 프레임을 재전송하는 것



DLC 서비스

■ 비연결형과 연결형

▶ 비연결형 프로토콜

- 프레임은 프레임들 사이에 어떠한 관계도 없이(각 프레임은 독립적이다) 하나의 노드에서 다음 노드로 전달
- 프레임들 사이에 연결이 없다는 것을 의미
- 데이터 링크 프로토콜들은 비연결형 프로토콜

▶ 연결형 프로토콜

- 연결 설정 단계, 전송 단계, 연결 해제 단계
- 점-대-점 프로토콜, 무선 LAN 및 WAN에서 사용

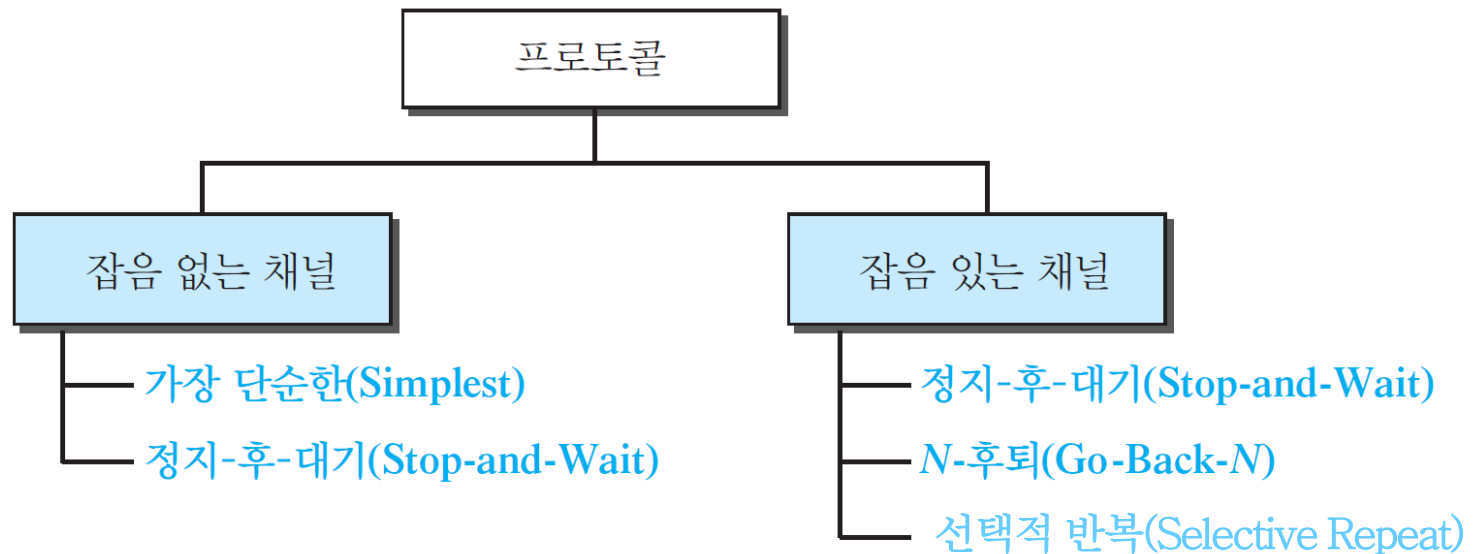
프로토콜

■ 프로토콜

- ▶ 한 노드에서 다른 노드로 데이터를 보내기 위해 어떻게 프레임 짜기, 흐름 제어와 오류 제어를 수행하는 규칙
- ▶ 보통 프로그램 언어를 사용하여 소프트웨어로 구현
- ▶ 각 프로토콜은 의사 코드를 사용하여 설명
 - 구현 언어에 의존하지 않기 위해
 - 언어 규칙에 구애 받지 않고 절차 자체를 설명하기 위해

프로토콜

▶ 프로토콜 종류



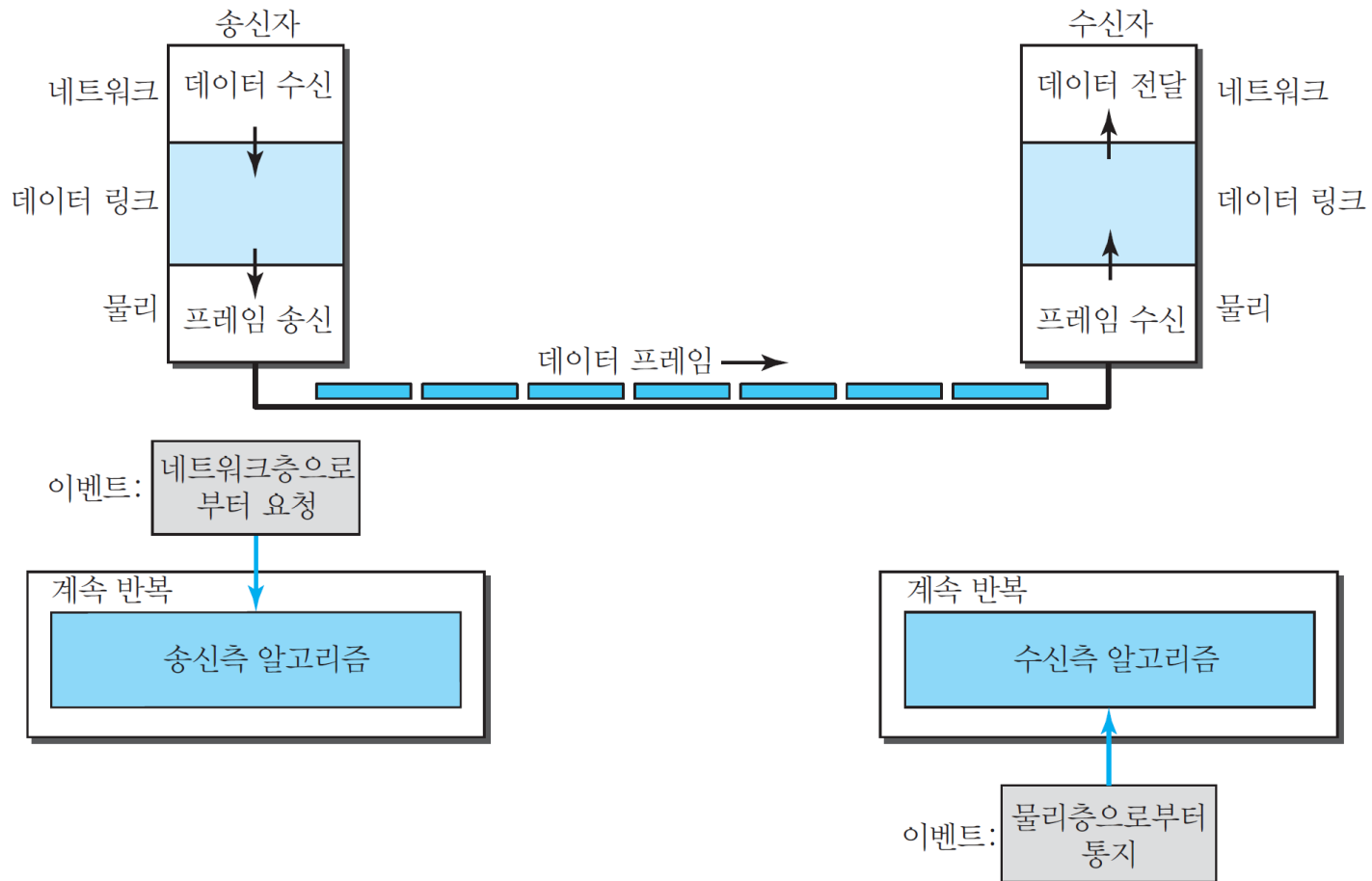
데이터 링크층 프로토콜

■ 잡음 없는 채널

- ▶ 우선 프레임 손실도 없고 중복되지도 않으며 손상되지도 않는 이상적인 채널
- ▶ 오류 제어는 없음
- ▶ 단순 프로토콜(simple protocol)
- ▶ 정지 후 대기 프로토콜(Stop and Wait protocol)

■ 단순 프로토콜

- ▶ 흐름 제어나 오류 제어를 하지 않는 프로토콜



데이터 링크층 프로토콜

▶ 알고리즘-송신자 쪽 절차

```
1 while(true)                                // 계속 반복
2 {
3     WaitForEvent();                          // 이벤트가 일어날 때까지 대기
4     if(Event(RequestToSend))                // 보내는 요청 이벤트 발생
5     {
6         GetData();
7         MakeFrame();
8         SendFrame();                          // 프레임 보내기
9     }
10 }
```

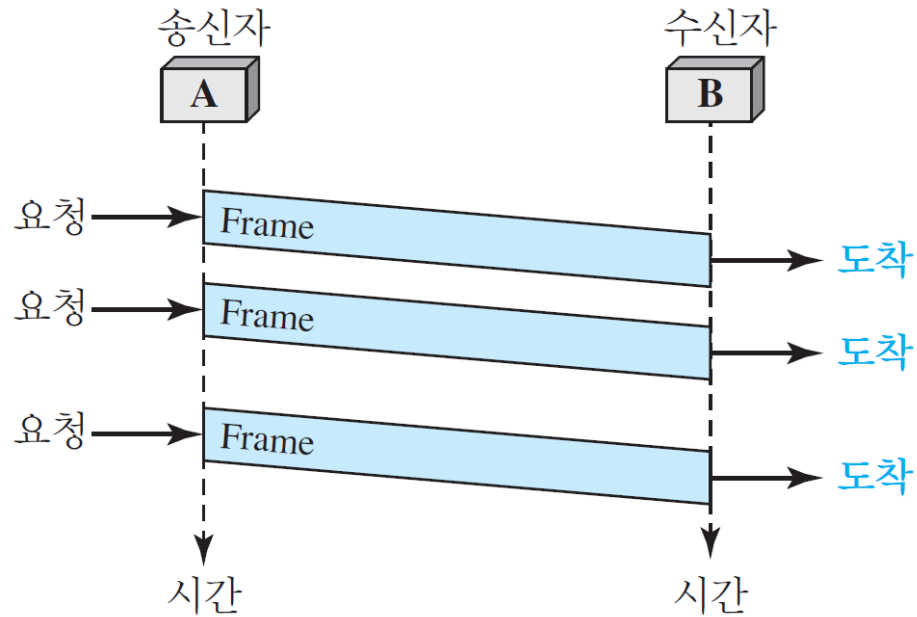
데이터 링크층 프로토콜

▶ 알고리즘-수신자 쪽 절차

```
1 while(true)                                // 계속 반복
2 {
3     WaitForEvent();                          // 이벤트가 생길 때까지 대기
4     if(Event(ArrivalNotification))           // 데이터 프레임 도착
5     {
6         ReceiveFrame();
7         ExtractData();
8         DeliverData();                       // 데이터를 네트워크층으로 전달
9     }
10 }
```

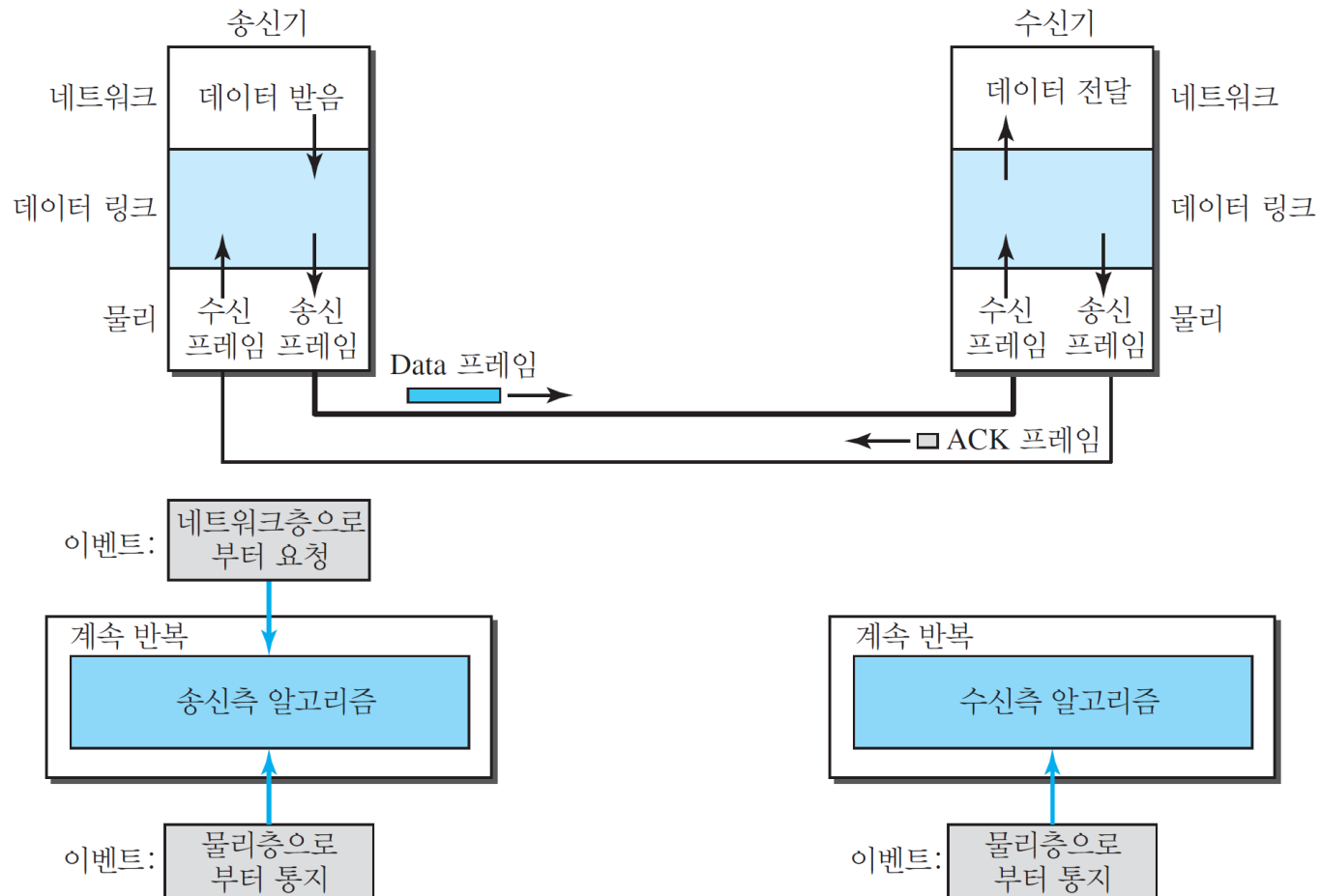

데이터 링크층 프로토콜

▶ 흐름도



■ 정지 후 대기 프로토콜(Stop and Wait protocol)

- ▶ 송신자가 한 개의 프레임을 전송한 후에 수신자로부터 확인 응답(ACK)을 받을 때까지 기다린 후에 다음 프레임을 전달



데이터 링크층 프로토콜

▶ 알고리즘-송신자측

```
1 while(true)                                //계속 반복
2   canSend = true                            //첫 번째 프레임을 보내는 것을 허용
3   {
4     WaitForEvent();                          // 이벤트가 발생할 때까지 대기
5     if(Event(RequestToSend) AND canSend)
6     {
7       GetData();
8       MakeFrame();
9       SendFrame();                          //데이터 프레임을 보낸다
10      canSend = false;                      //ACK가 도착될 때까지 보낼 수 없다
11    }
12    WaitForEvent();                          // 이벤트가 발생할 때까지 대기
13    if(Event(ArrivalNotification) // ACK 도착
14    {
15      ReceiveFrame();                       //ACK 프레임 수신
16      canSend = true;
17    }
18  }
```

데이터 링크층 프로토콜

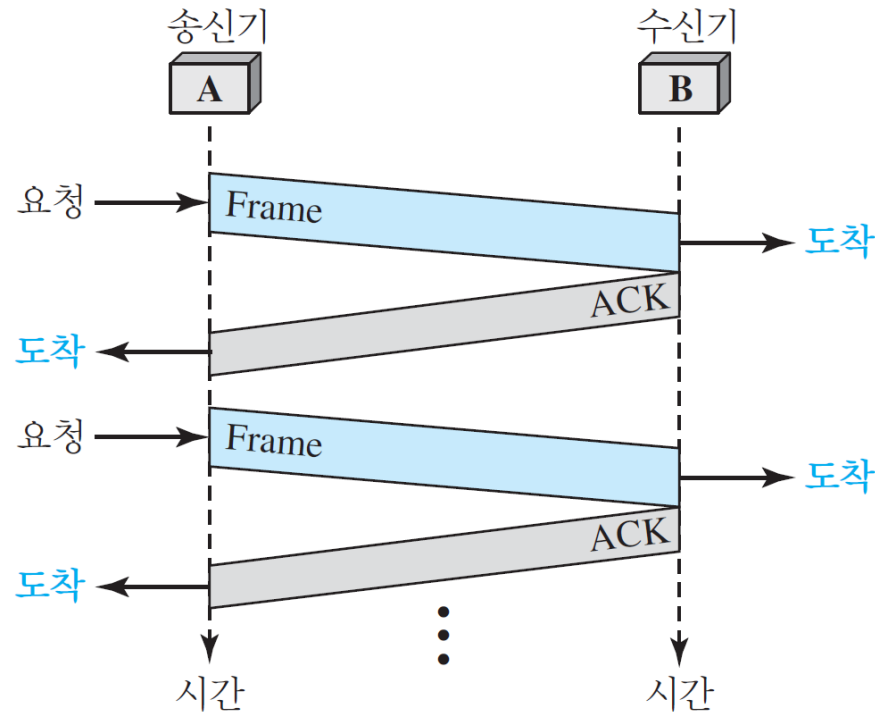
▶ 알고리즘-수신자측

```
1 while(true)                                //계속 반복
2 {
3     WaitForEvent();                          // 이벤트가 발생할 때까지 대기
4     if(Event(ArrivalNotification))           //데이터 프레임 도착
5     {
6         ReceiveFrame();
7         ExtractData();
8         Deliver(data);                       //데이터를 네트워크층으로 전달
9         SendFrame();                         //프레임 송신
10    }
11 }
```

데이터 링크층 프로토콜

▶ 흐름도

- 송신자는 프레임을 하나 보내고 수신자의 확인응답을 기다린다
- ACK가 도달하면 송신자는 다음 프레임을 전달



데이터 링크층 프로토콜

■ 잡음 있는 채널

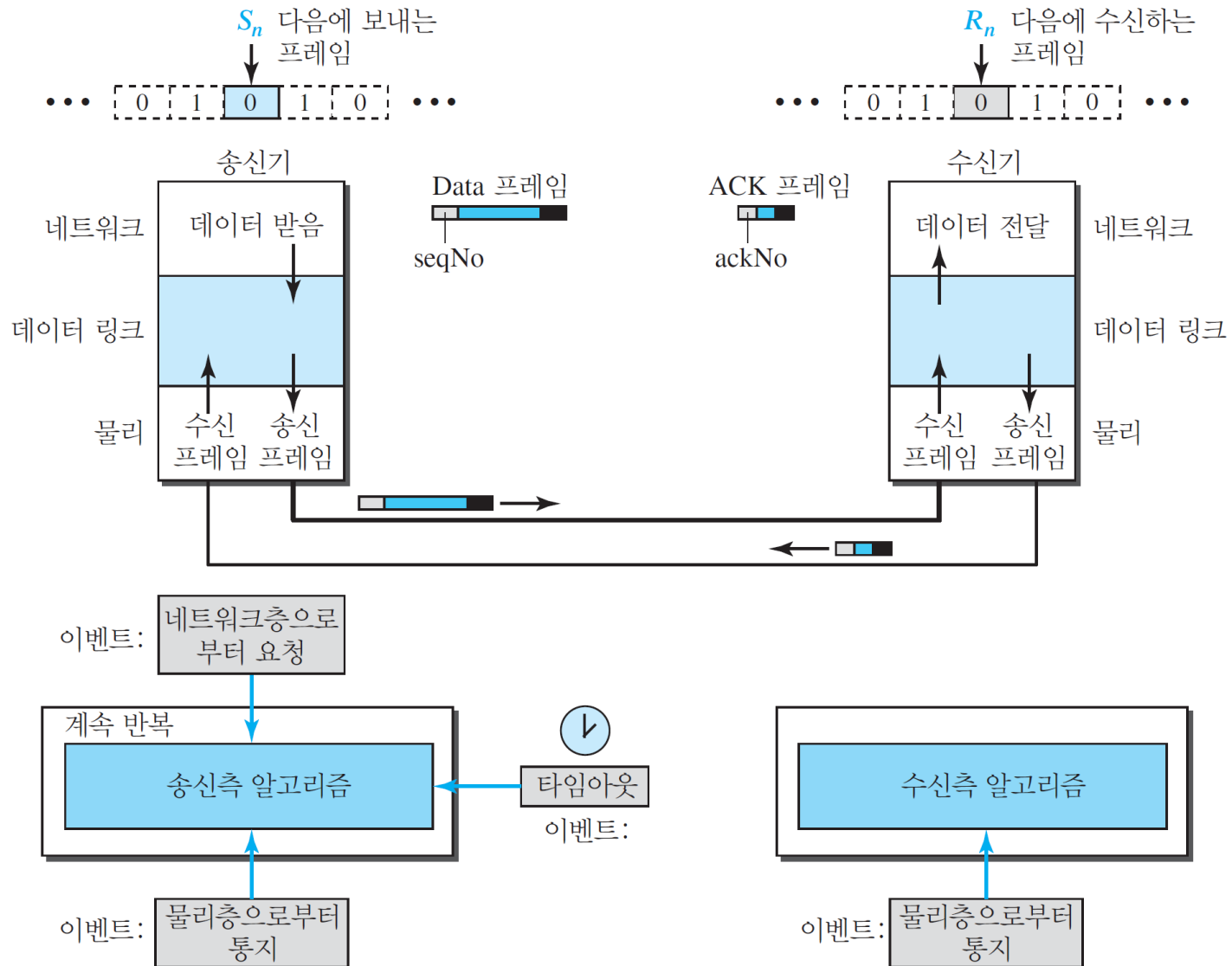
- ▶ 오류 제어를 사용하는 세 개의 프로토콜
- ▶ ARQ(Automatic Repeat Request)
- ▶ 정지-후-대기 ARQ(Stop-and-Wait ARQ)
- ▶ N-복귀 ARQ(Go-Back-N ARQ)
- ▶ 선택적 반복 ARQ(selective repeat ARQ)

데이터 링크층 프로토콜

■ 정지-후-대기 ARQ

- ▶ 오류 제어는 전송된 프레임의 사본을 보관하고 있다가 타이머가 만료되면 해당 프레임을 다시 전달
- ▶ 순서 번호는 프레임에 부여하여 사용. 순서 번호는 모듈러-2 연산을 기반
- ▶ 확인응답 번호는 다음에 받기를 기대하는 프레임의 번호를 모듈러-2 연산으로 만들어 전달

▶ 정지-후-대기 ARQ 프로토콜의 설계



▶ 알고리즘-송신자측

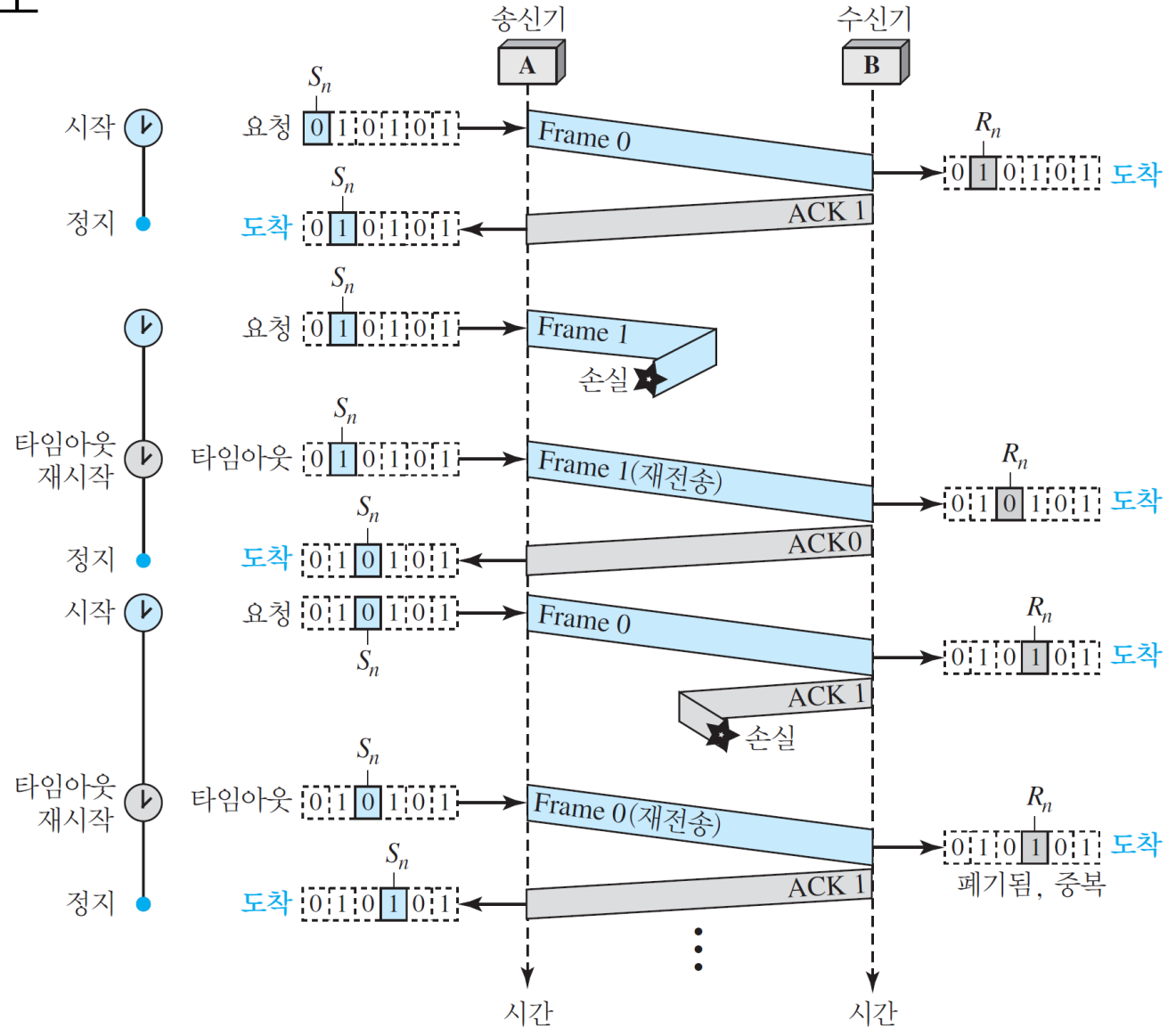
```
1  Sn = 0; // Frame 0을 첫번째로 보낸다
2  canSend = true; // 첫 번째 요청 허용
3  while(true) // 계속 반복
4  {
5      WaitForEvent(); // 이벤트가 발생할 때까지 대기
6      if(Event(RequestToSend) AND canSend)
7      {
8          GetData();
9          MakeFrame(Sn); //seqNo는 Sn
10         StoreFrame(Sn); //복사해 둔다
11         SendFrame(Sn);
12         StartTimer();
13         Sn = Sn + 1;
14         canSend = false;
15     }
16     WaitForEvent(); // 대기
17     if(Event(ArrivalNotification) // ACK 도착
18     {
19         ReceiveFrame(ackNo); //ACK 프레임 수신
20         if(not corrupted AND ackNo == Sn) //정확한 ACK
21         {
22             Stoptimer();
23             PurgeFrame(Sn-1); //복사가 필요없음
24             canSend = true;
25         }
26     }
27
28     if(Event(TimeOut) // 타이머 만료
29     {
30         StartTimer();
31         ResendFrame(Sn-1); //복사해 둔 프레임 재전송
32     }
33 }
```

데이터 링크층 프로토콜

▶ 알고리즘-수신자측

```
1  Rn = 0;                                // 첫 번째 도착을 기대하는 0번 프레임
2  while(true)
3  {
4      WaitForEvent();                      // 이벤트가 발생할 때까지 대기
5      if(Event(ArrivalNotification))      //데이터 프레임 도착
6      {
7          ReceiveFrame();
8          if(corrupted(frame));
9          sleep();
10         if(seqNo == Rn)                  //올바른 데이터 프레임
11         {
12             ExtractData();
13             DeliverData();                //데이터 전달
14             Rn = Rn + 1;
15         }
16         SendFrame(Rn);                  //ACK 보냄
17     }
18 }
```

▶ 흐름도



데이터 링크층 프로토콜

■ N-복귀 ARQ

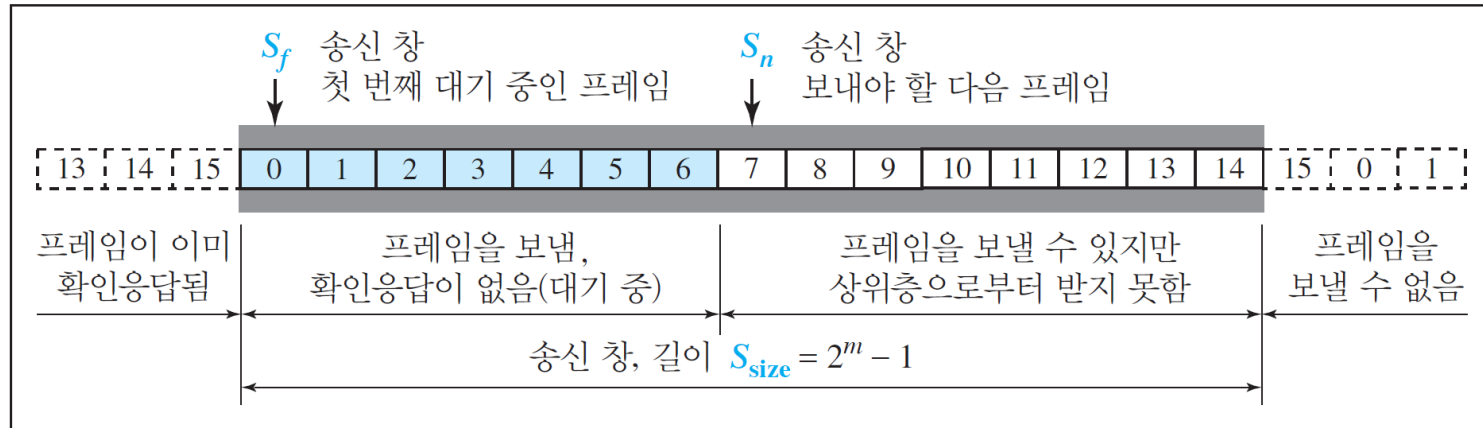
- ▶ 전송 효율을 높이기 위해 확인응답을 기다리는 동안 여러 개의 프레임을 전송하는 방식
- ▶ 순서번호
 - $0 \sim 2^m - 1$
 - 순서번호는 모듈로 2^m 인데, 여기서 m 은 비트 단위의 순서 번호 필드의 길이

데이터 링크층 프로토콜

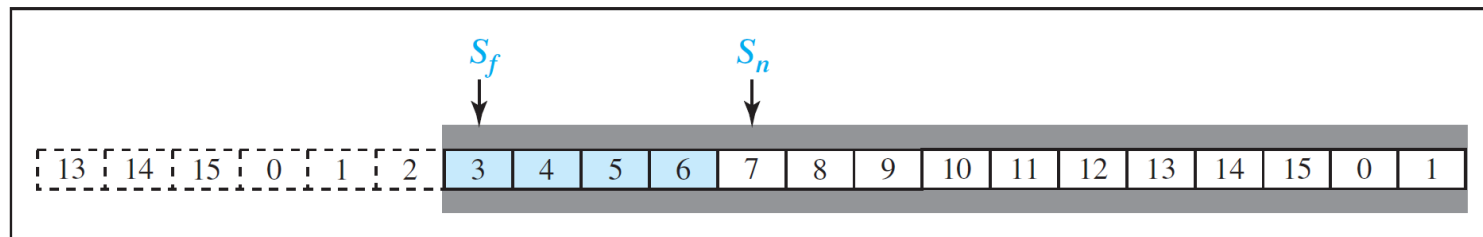
▶ 미닫이 창

- sliding window, 슬라이딩 윈도우
- 송신자와 수신자에게 필요한 순서 번호의 범위를 지정하는 추상적인 개념
- 송신 미닫이 창(send sliding window): 송신자에게 필요한 범위
 - ✓ 현재 전송할 수 있는 데이터 프레임의 순서 번호가 들어있는 가상의 상자
 - ✓ 세 개의 변수를 갖는 크기가 2^m-1 인 가상의 상자
 - » S_f : 1번째 미해결 프레임
 - » S_n : 전송할 다음 프레임
 - » S_{size} : 창의 크기

데이터 링크층 프로토콜



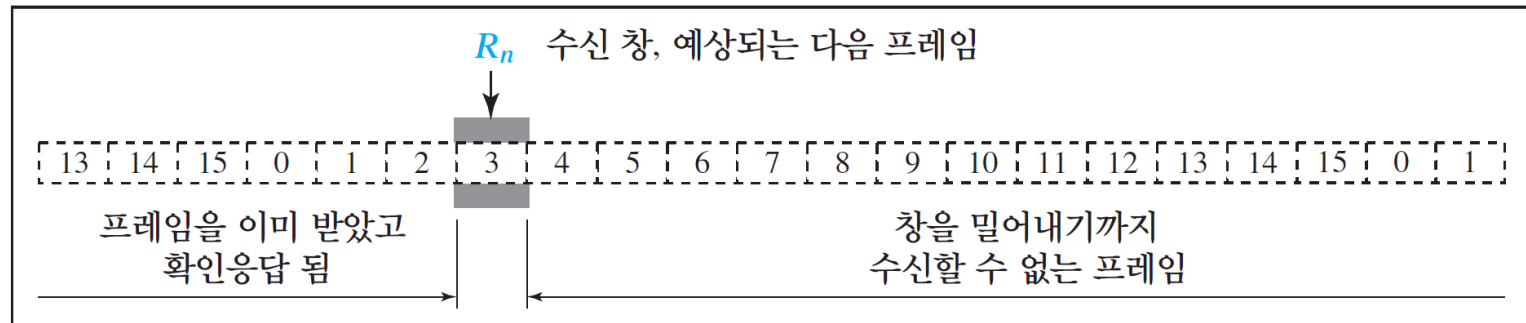
a. 밀어내기 전 송신 창



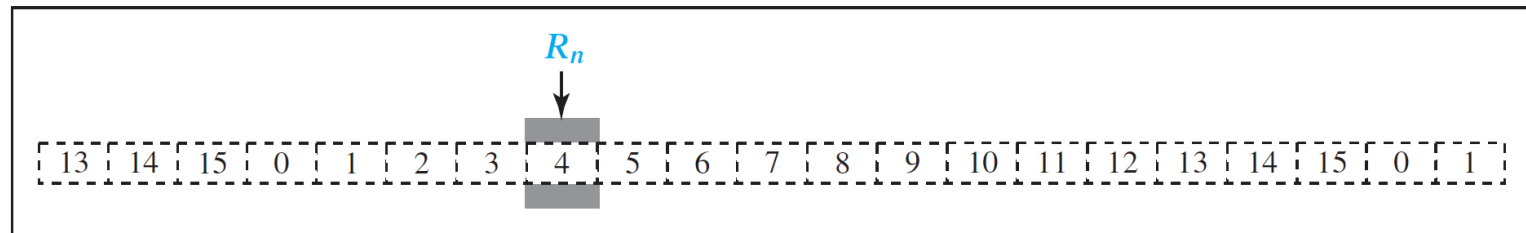
b. 밀어낸 후 송신 창

데이터 링크층 프로토콜

- 수신 미닫이 창(receive sliding window): 수신자가 필요한 범위
 - ✓ 데이터 프레임이 정확하게 수신되었고 정확한 확인응답을 전송했는지를 확인
 - ✓ 한 개의 변수 R_n 을 갖는 크기가 1인 가상 상자

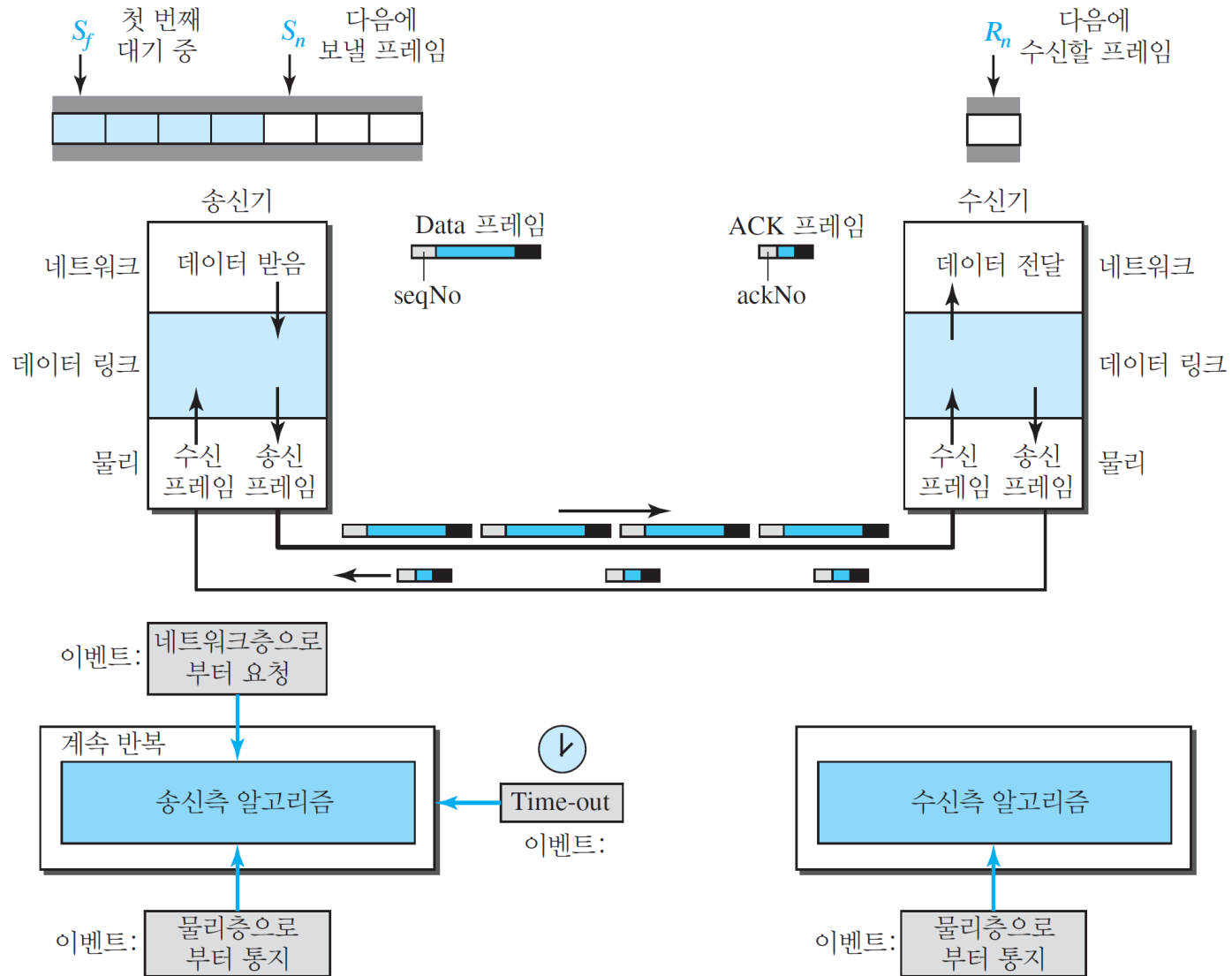


a. 수신 창



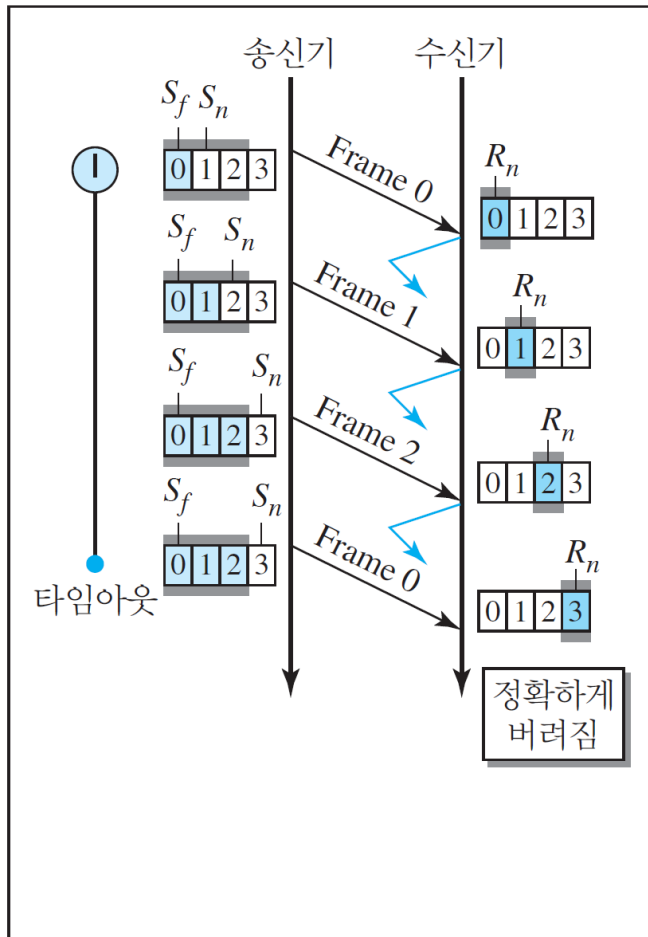
b. 밀어낸 후의 창

▶ N-복귀 ARQ의 설계

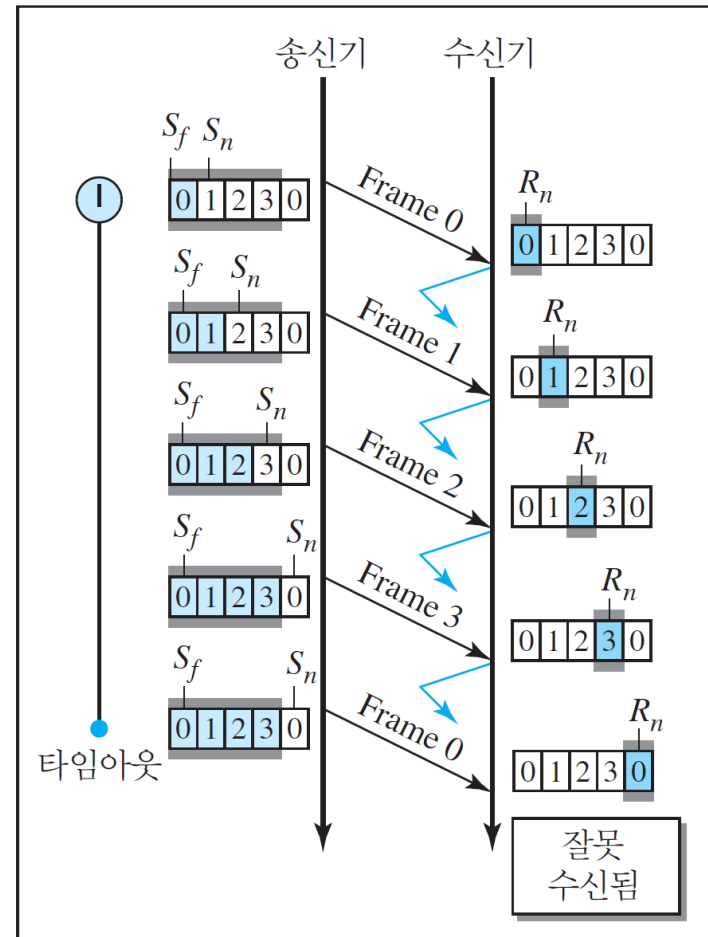


▶ N-복귀 ARQ의 창 크기

- 송신 창의 크기는 2^m 보다 작고 수신 창의 크기는 항상 1



a. 창 크기 $< 2^m$



b. 창 크기 $= 2^m$

데이터 링크층 프로토콜

▶ 알고리즘-송신자 측

```
1   $S_w = 2^m - 1$ ;  
2   $S_f = 0$ ;  
3   $S_n = 0$ ;  
4  
5  while (true)                                //계속 반복  
6  {  
7      WaitForEvent();  
8      if(Event(RequestToSend))                //패킷을 보낸다  
9      {  
10         if( $S_n - S_f \geq S_w$ )                 //창이 꽉 차있으면  
11             Sleep();  
12         GetData();  
13         MakeFrame( $S_n$ );  
14         StoreFrame( $S_n$ );  
15         SendFrame( $S_n$ );  
16          $S_n = S_n + 1$ ;  
17         if(timer not running)  
18             StartTimer();  
19     }  
20
```

```

20
21  if(Event(ArrivalNotification))  //ACK 도착
22  {
23      Receive(ACK);
24      if(corrupted(ACK))
25          Sleep();
26      if((ackNo>Sf)&&(ackNo<=Sn))  //올바른 ACK이면
27      While(Sf <= ackNo)
28      {
29          PurgeFrame(Sf);
30          Sf = Sf + 1;
31      }
32      StopTimer();
33  }
34
35  if(Event(TimeOut))  //타이머 만료
36  {
37      StartTimer();
38      Temp = Sf;
39      while(Temp < Sn);
40      {
41          SendFrame(Sf);
42          Sf = Sf + 1;
43      }
44  }
45  }

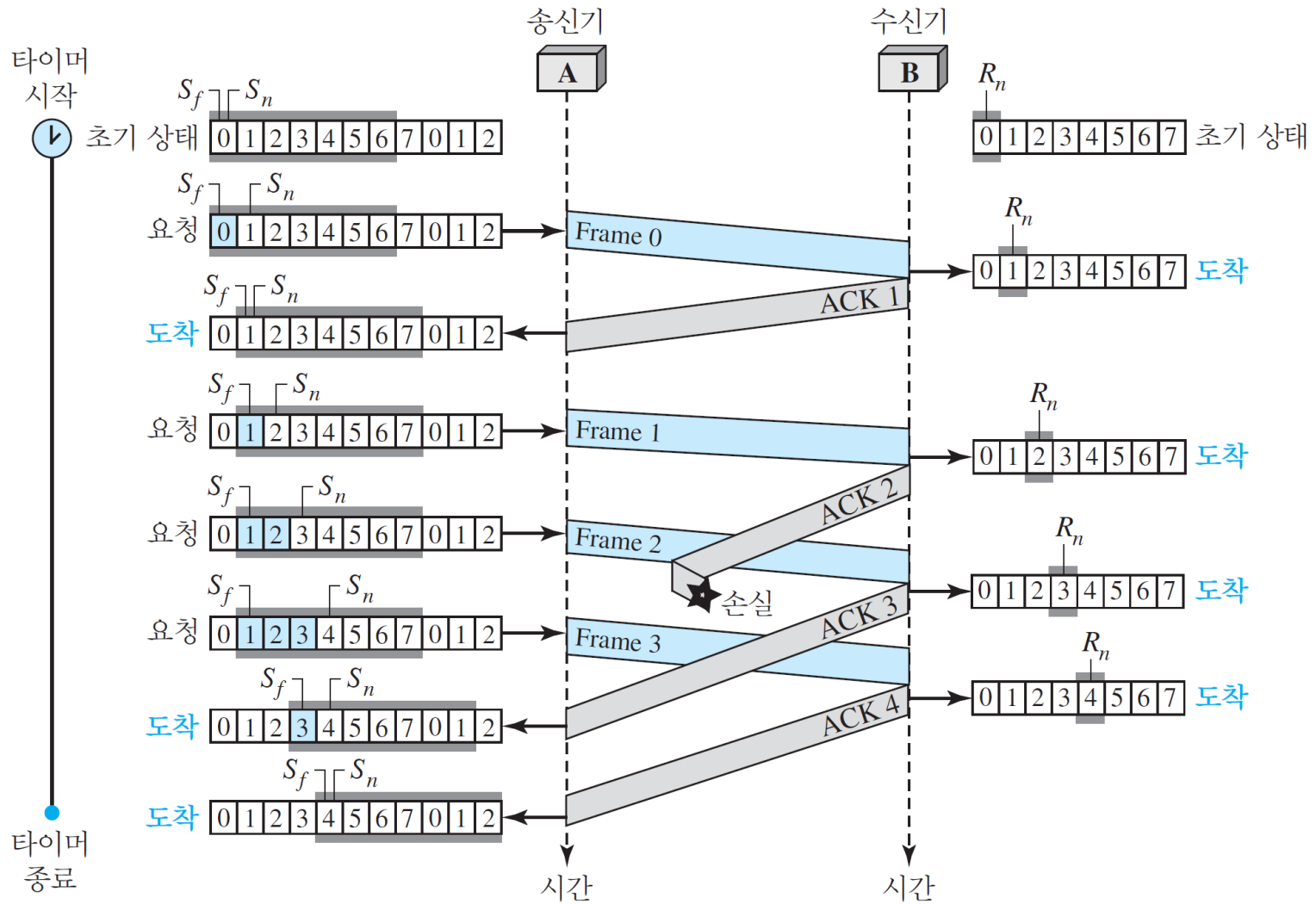
```

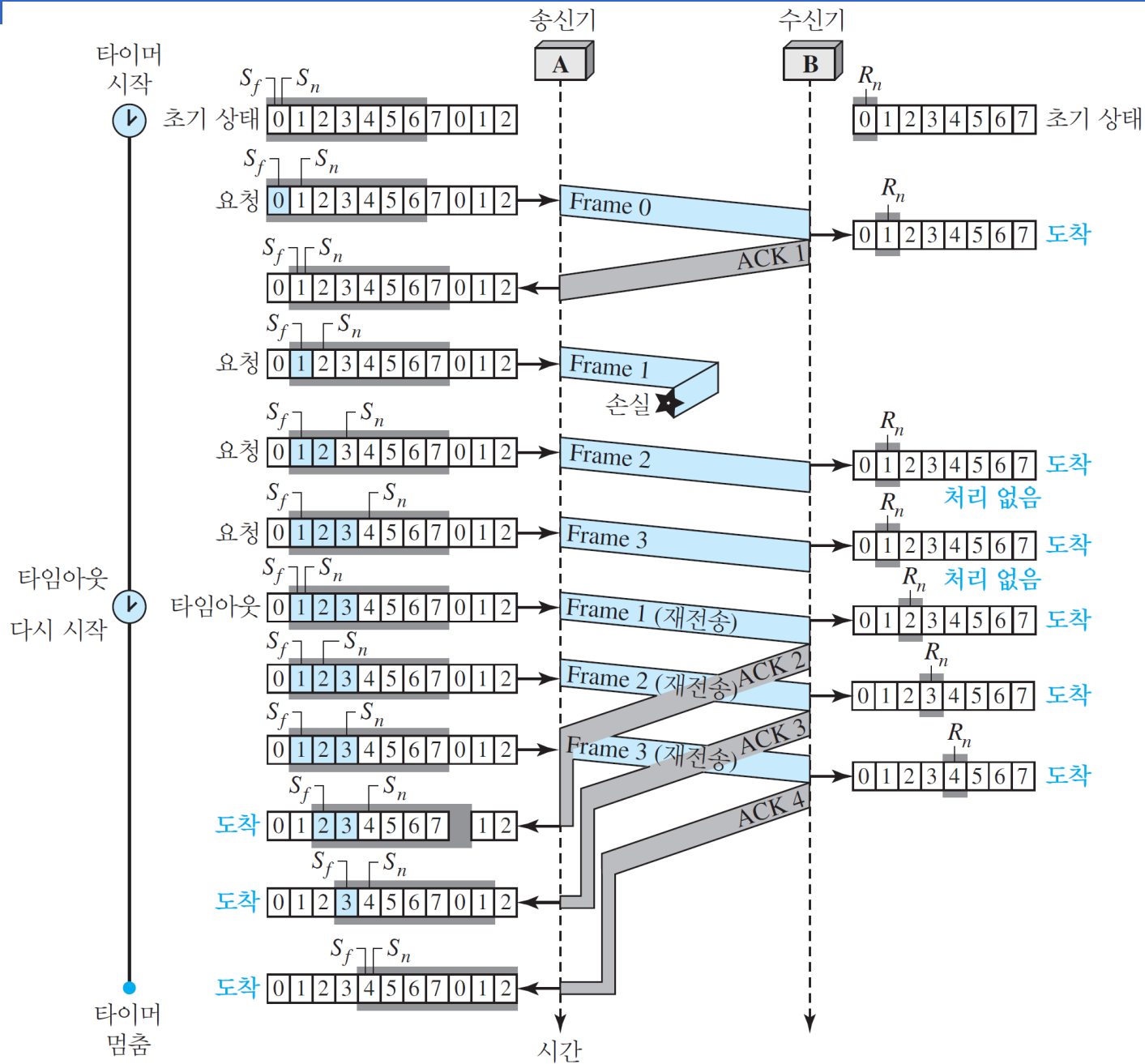
데이터 링크층 프로토콜

▶ 알고리즘-수신자 측

```
1 Rn = 0;
2
3 while (true)                                //계속 반복
4 {
5     WaitForEvent();
6
7     if(Event(ArrivalNotification))           //데이터 프레임 도착
8     {
9         Receive(Frame);
10        if(corrupted(Frame))
11            Sleep();
12        if(seqNo == Rn)                       //예상한 프레임이면
13        {
14            DeliverData();                     //데이터 전달
15            Rn = Rn + 1;                       //창을 한 칸 밀어낸다
16            SendACK(Rn);
17        }
18    }
19 }
```

▶ 흐름도



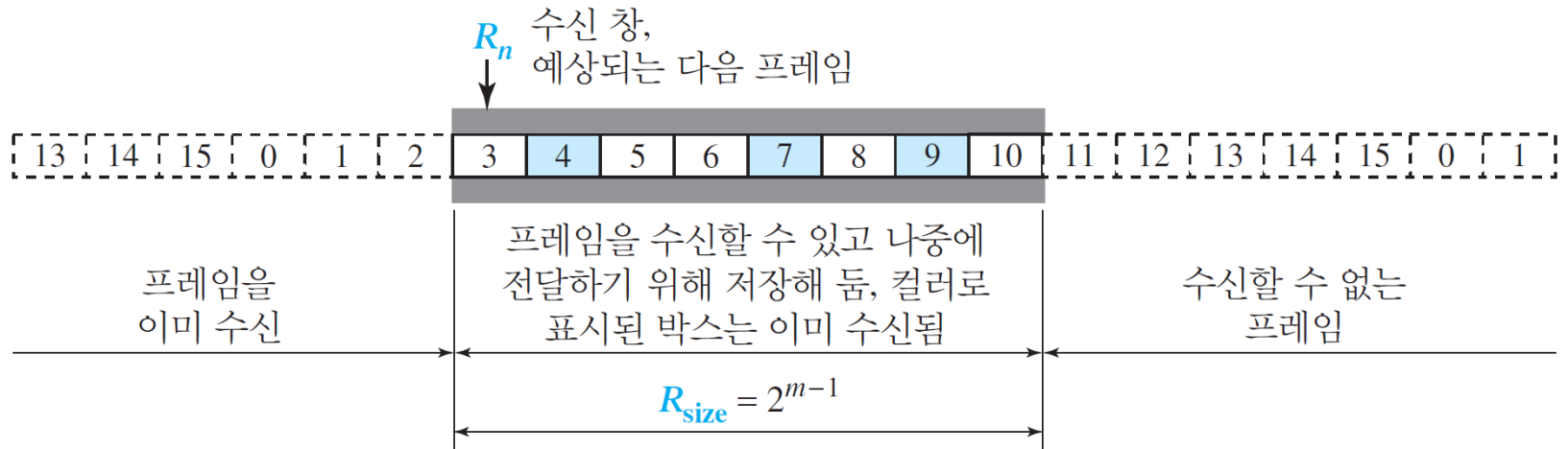
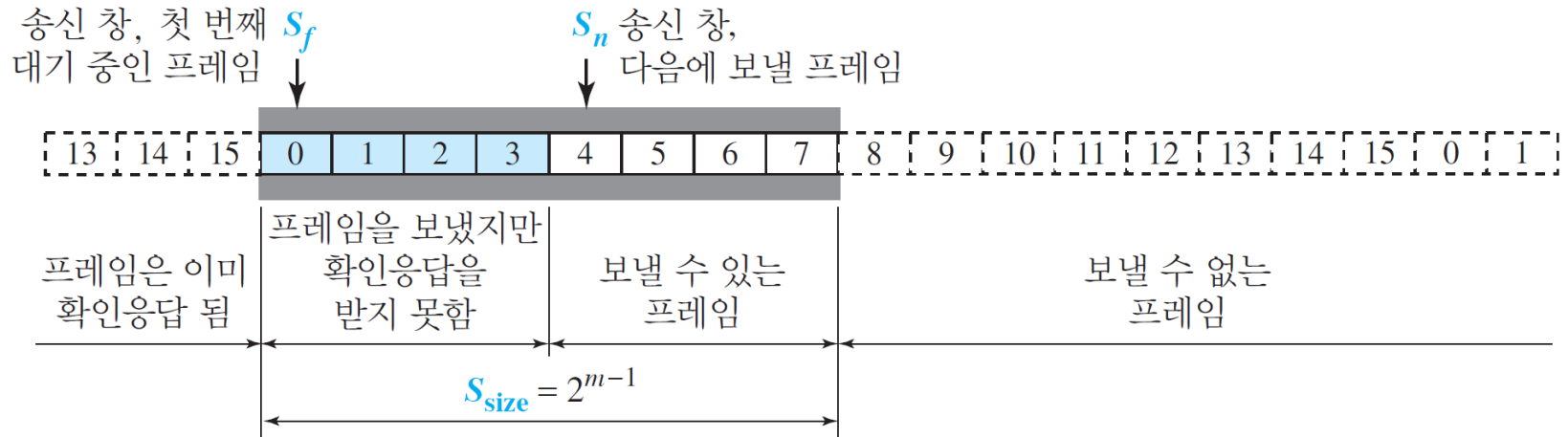


데이터 링크층 프로토콜

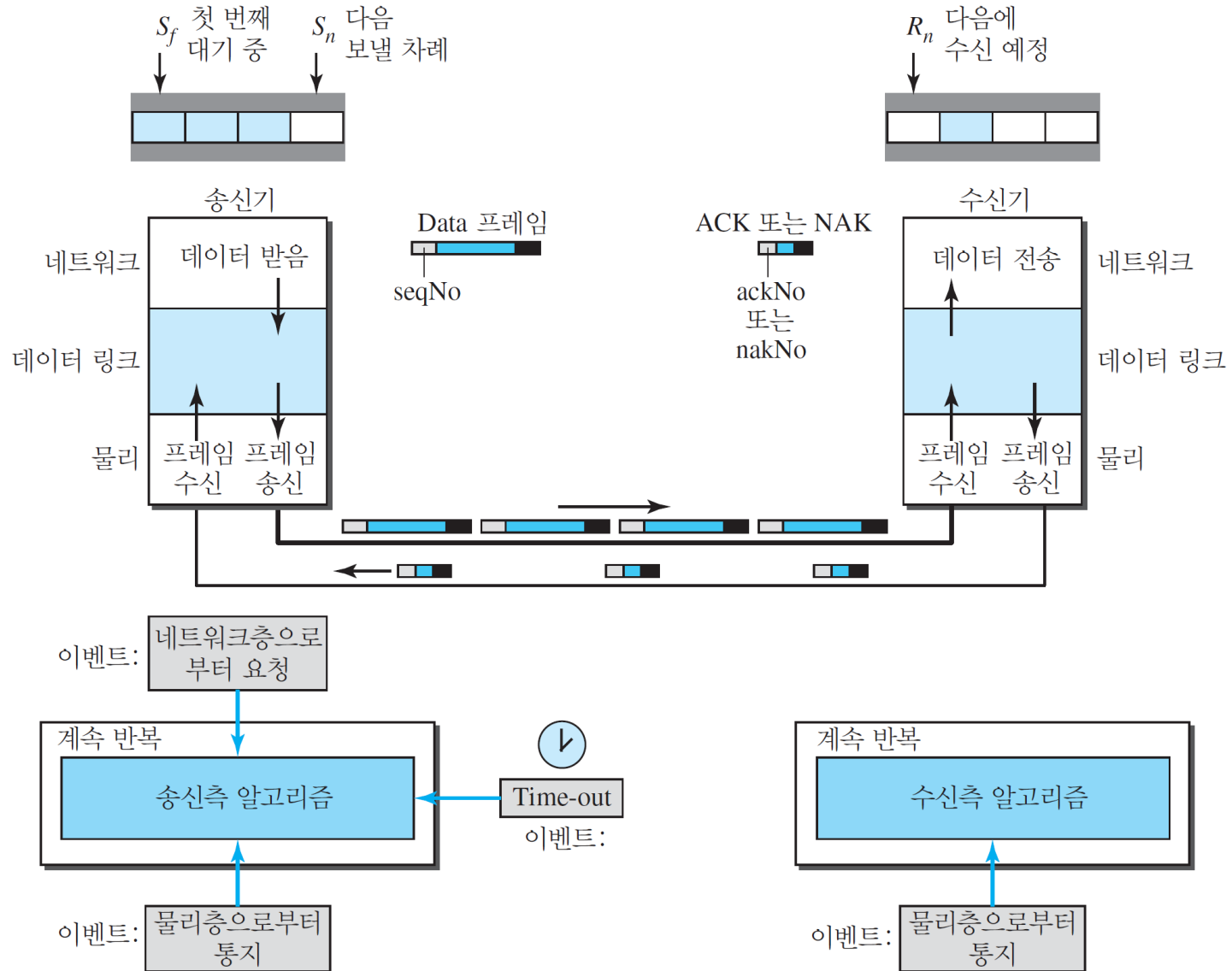
■ 선택적 반복 ARQ

- ▶ 전체 N개의 프레임을 전달하는 대신 손상된 프레임만 전송하는 방식
- ▶ 잡음이 많은 채널에서 효율적
- ▶ 수신자 측의 처리절차가 복잡해짐

▶ 선택적 반복 ARQ의 송신과 수신 창

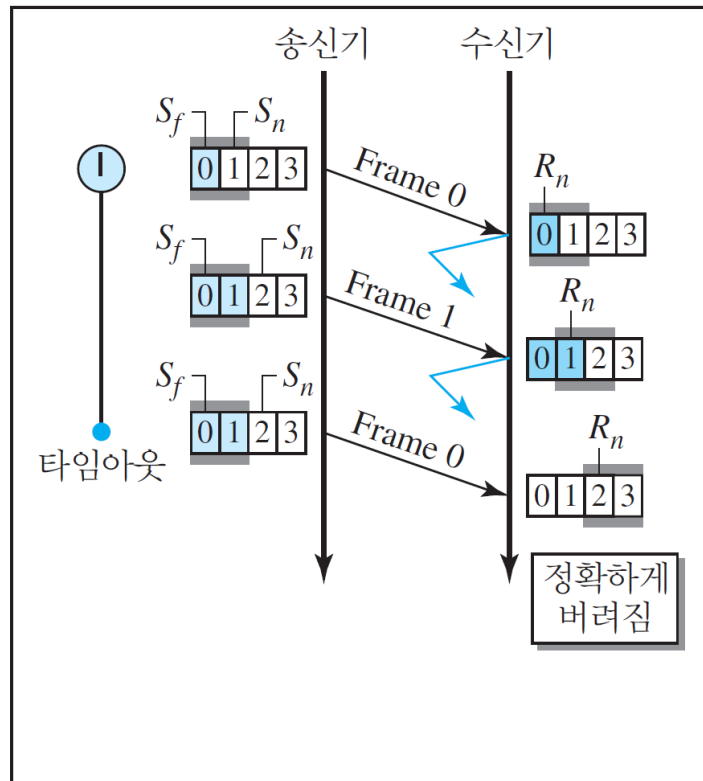


▶ 선택적 반복 ARQ의 설계

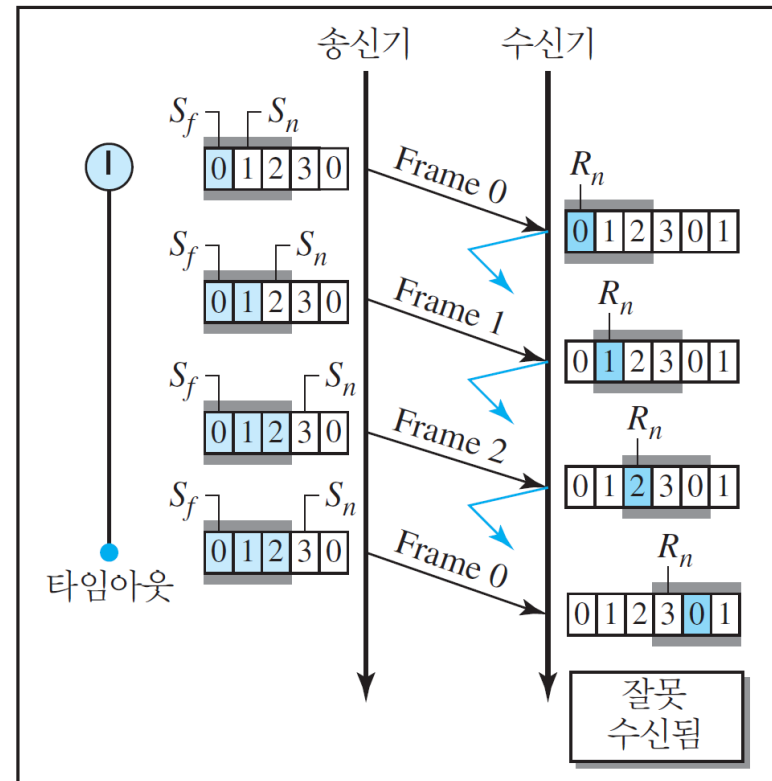


잡음 있는 채널

- ▶ 선택적 반복 ARQ 창 크기
 - 송신 창과 수신 창의 크기는 최대 2^m 의 절반



a. 창 크기 = 2^{m-1}



b. 창 크기 > 2^{m-1}

잡음 있는 채널

▶ 알고리즘-송신자 측

```
1   $S_w = 2^{m-1}$  ;
2   $S_f = 0$ ;
3   $S_n = 0$ ;
4
5  while (true)                                //계속 반복
6  {
7      WaitForEvent();
8      if(Event(RequestToSend))                //패킷을 보낸다
9      {
10         if( $S_n - S_f \geq S_w$ )                  //창이 꽉 차있으면
11             Sleep();
12         GetData();
13         MakeFrame( $S_n$ );
14         StoreFrame( $S_n$ );
15         SendFrame( $S_n$ );
16          $S_n = S_n + 1$ ;
17         StartTimer( $S_n$ );
18     }
19
```

```

19
20   if(Event(ArrivalNotification)) //ACK 도착
21   {
22       Receive(frame);           //ACK 또는 NAK 수신
23       if(corrupted(frame))
24           Sleep();
25       if (FrameType == NAK)
26           if (nakNo between  $S_f$  and  $S_n$ )
27           {
28               resend(nakNo);
29               StartTimer(nakNo);
30           }
31       if (FrameType == ACK)
32           if (ackNo between  $S_f$  and  $S_n$ )
33           {
34               while( $s_f < \text{ackNo}$ )
35               {
36                   Purge( $s_f$ );
37                   StopTimer( $s_f$ );
38                    $S_f = S_f + 1$ ;
39               }
40           }
41   }
42
43   if(Event(TimeOut(t))) //타이머 만료
44   {
45       StartTimer(t);
46       SendFrame(t);
47   }
48 }

```

데이터 링크층 프로토콜

▶ 알고리즘-수신자 측

```
1  Rn = 0;
2  NakSent = false;
3  AckNeeded = false;
4  Repeat(for all slots)
5      Marked(slot) = false;
6
7  while (true)                                //계속 반복
8  {
9      WaitForEvent();
10
11     if(Event(ArrivalNotification))           /데이터 프레임 도착
12     {
13         Receive(Frame);
14         if(corrupted(Frame)) && (NOT NakSent)
15         {
16             SendNAK(Rn);
17             NakSent = true;
18             Sleep();
19         }
```

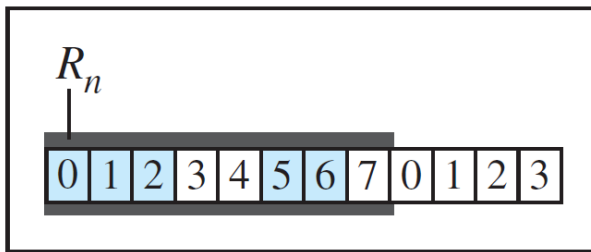
```

20     if(seqNo <> Rn)&& (NOT NakSent)
21     {
22         SendNAK(Rn);
23         NakSent = true;
24         if ((seqNo in window)&&(!Marked(seqNo)))
25         {
26             StoreFrame(seqNo)
27             Marked(seqNo)= true;
28             while(Marked(Rn))
29             {
30                 DeliverData(Rn);
31                 Purge(Rn);
32                 Rn = Rn + 1;
33                 AckNeeded = true;
34             }
35             if(AckNeeded);
36             {
37                 SendAck(Rn);
38                 AckNeeded = false;
39                 NakSent = false;
40             }
41         }
42     }
43 }
44 }

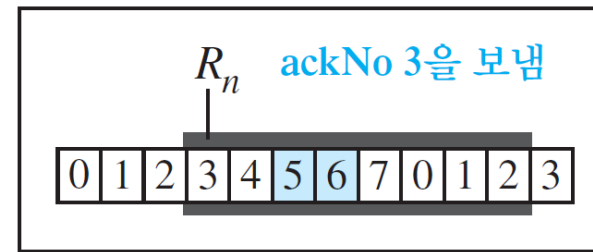
```

데이터 링크층 프로토콜

▶ 선택적 반복 ARQ의 데이터 전달

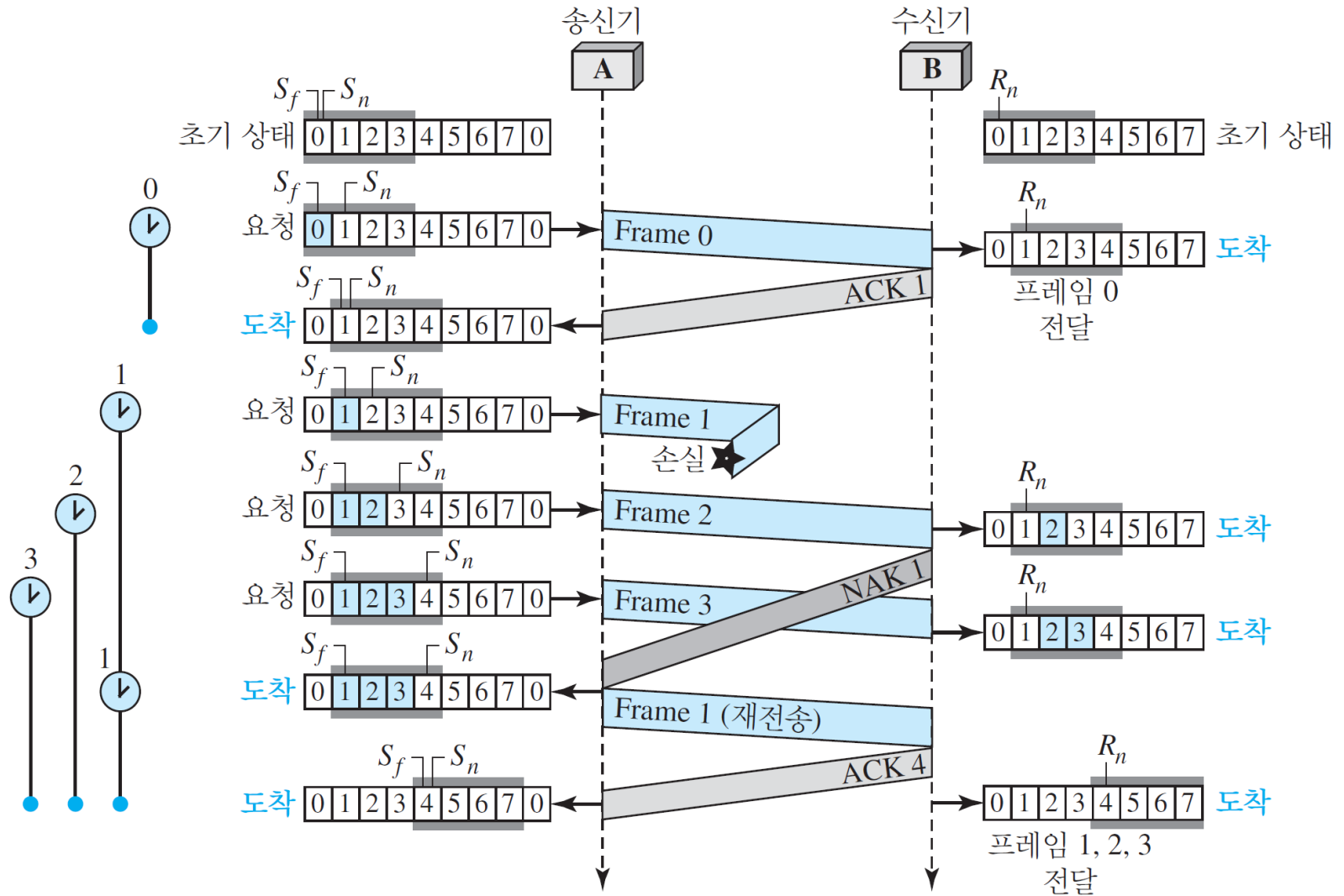


a. 전달 전



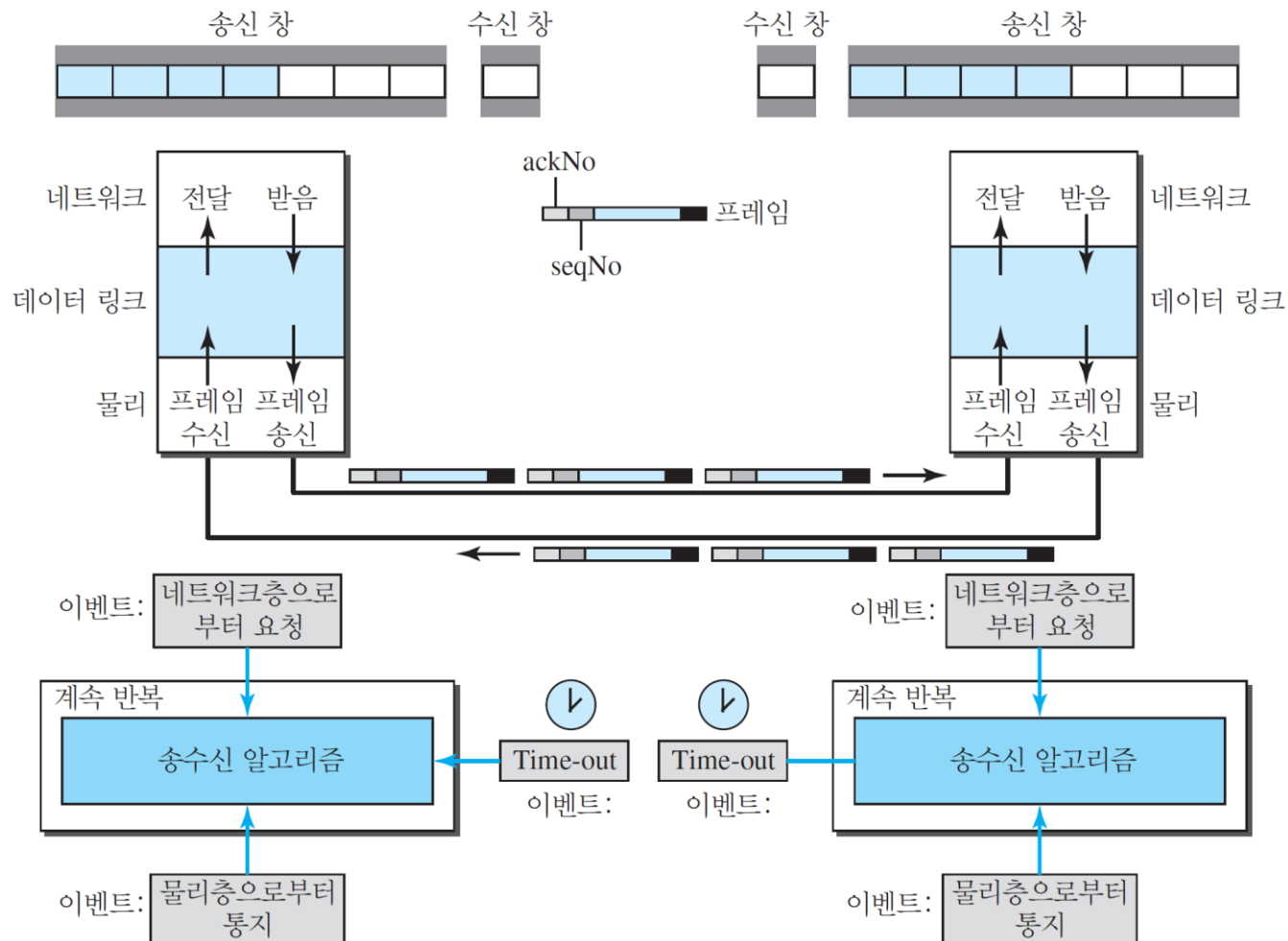
b. 전달 후

▶ 흐름도



■ 피기백킹(Piggybacking)

- ▶ 양방향 프로토콜의 효율을 높이기 위해 데이터 프레임에 제어 정보를 포함해서 보내는 기술



HDLC

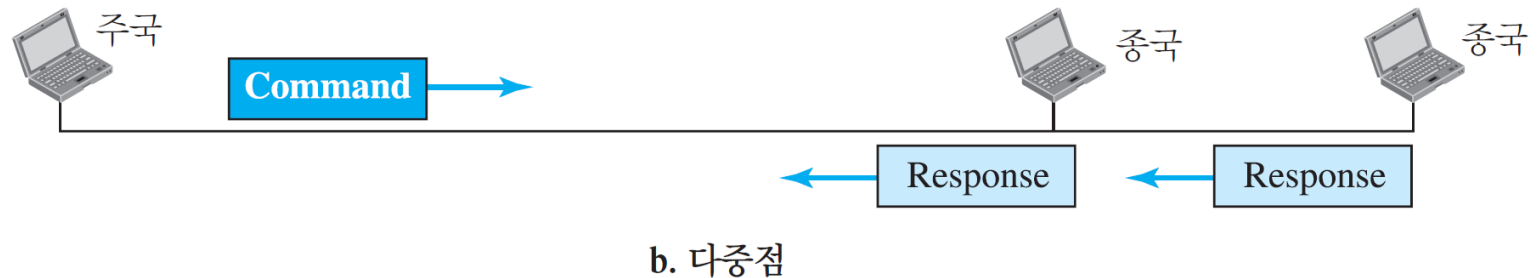
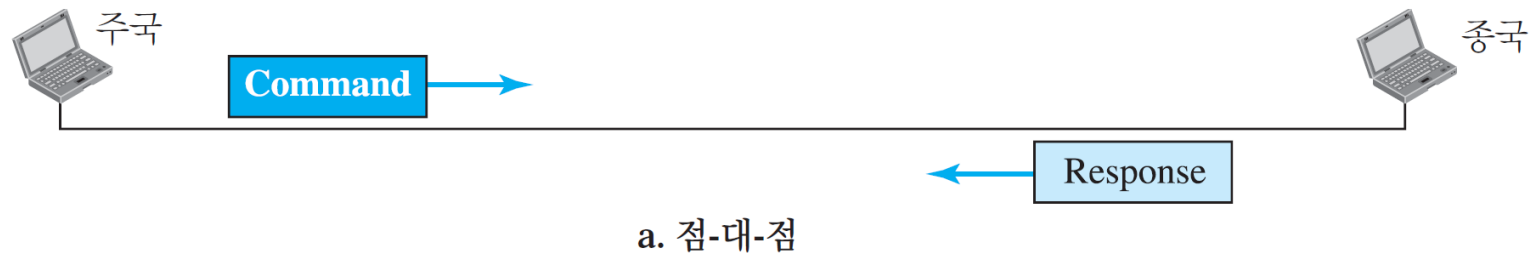
- HDLC(High-level Data Link Control)
 - ▶ 고급 데이터 링크 제어
 - ▶ 점-대-점과 다중점 링크에서 통신을 위한 비트 중심 프로토콜
 - ▶ 반이중 통신과 전이중 통신을 지원하도록 설계

HDLC

■ 구성과 전송 모드

▶ 정규 응답 모드

- 주국(primary station): 명령 전송
- 종국(secondary station): 응답 전송



HDLC

▶ 비동기 균형 모드



HDLC

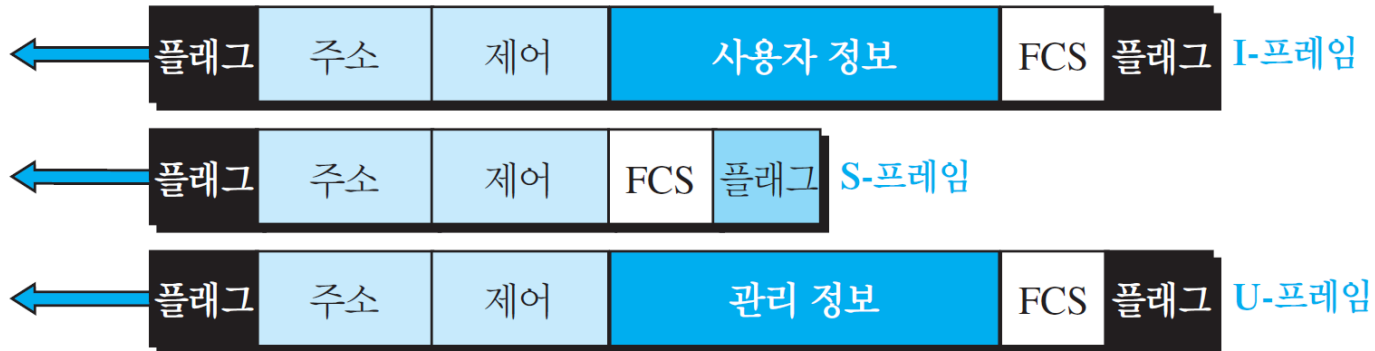
■ 프레임

- ▶ 정보 프레임(I-프레임, information frames),
 - 사용자 데이터 전달용
- ▶ 감시 프레임(S-프레임, supervisory frames)
 - 흐름제어와 오류제어용
- ▶ 비번호프레임(U-프레임, unnumbered frames)
 - 세션관리와 제어정보 교환용

HDLC

▶ 필드

- 플래그 필드: 01111110
- 주소 필드
- 제어 필드
- 정보 필드
- FCS(Frame Check Sequence) 필드



HDLC

▶ 제어 필드

- I-프레임

- ✓ N(S): 전송 중인 프레임 순서 번호

- ✓ N(R): ACK 번호

- ✓ P/F: Poll(주국 → 종국), Final(종국 → 주국)

- S-프레임

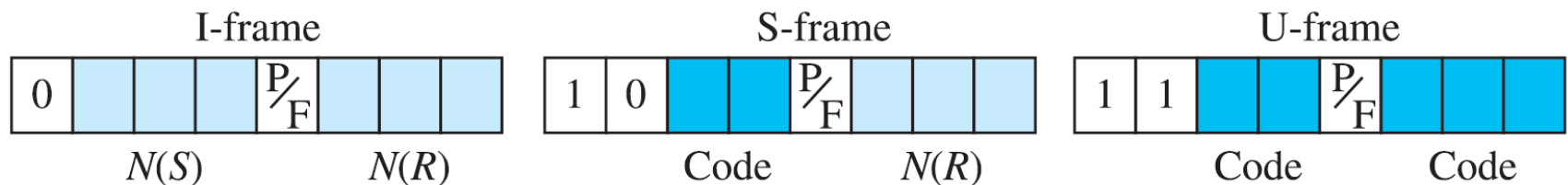
- ✓ Code

- » 수신 준비(RR, Receive Ready), 00

- » 수신 불가(RNR, Receive Not Ready), 10

- » 거부(REJ, Reject), 01

- » 선택적 거부(SREJ, Selective Reject), 11

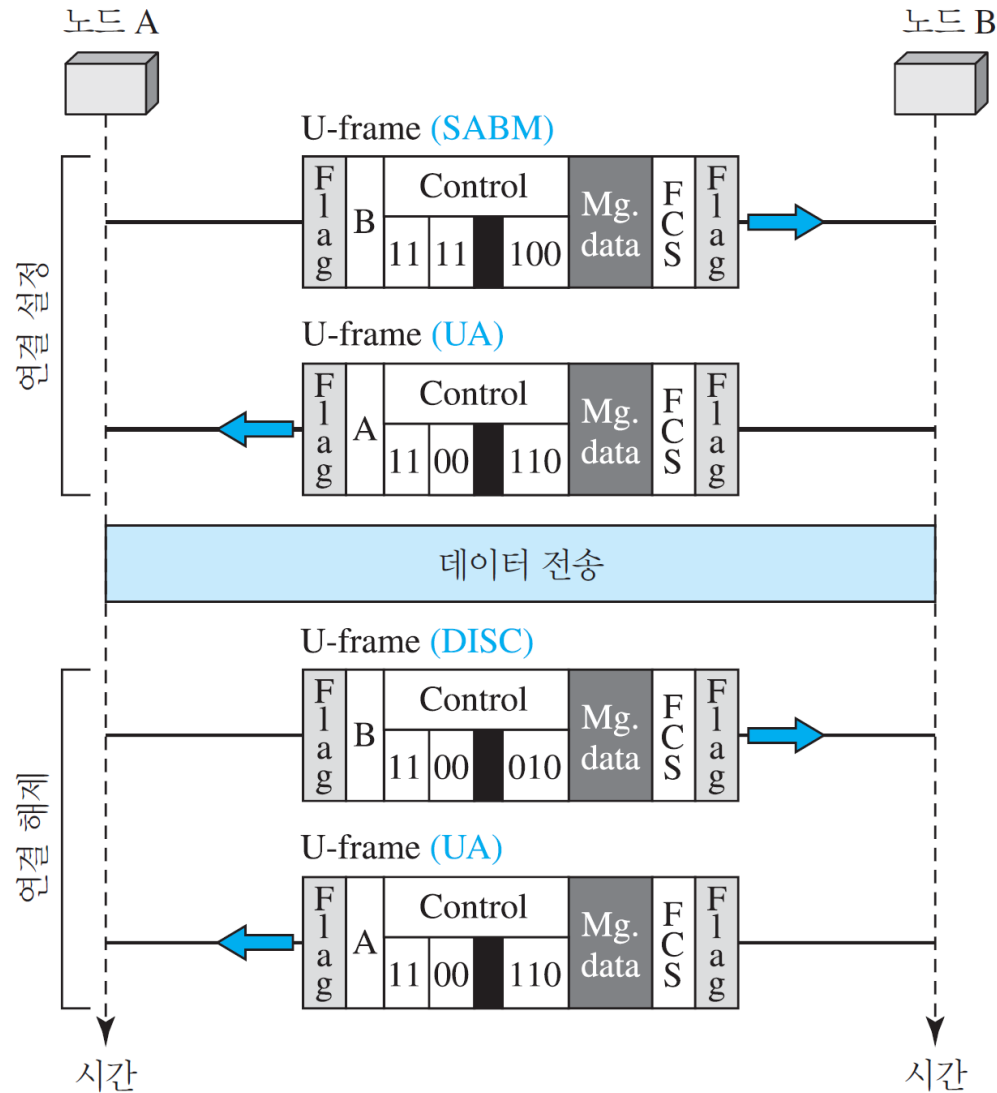


- U-프레임 코드

코드	명령	응답	의미
00 001	SNRM		set normal response mode
11 011	SNRME		set normal response mode, extended
11 100	SABM	DM	set asynchronous balanced mode 또는 disconnect mode
11 110	SABME		set asynchronous balanced mode, extended
00 000	UI	UI	Unnumbered information
00 110		UA	Unnumbered acknowledgment
00 010	DISC	RD	Disconnect 또는 request disconnect
10 000	SIM	RIM	set initialization mode 또는 request information mode
00 100	UP		Unnumbered poll
11 001	RSET		Reset
11 101	XID	XID	Exchange ID
10 001	FRMR	FRMR	Frame reject

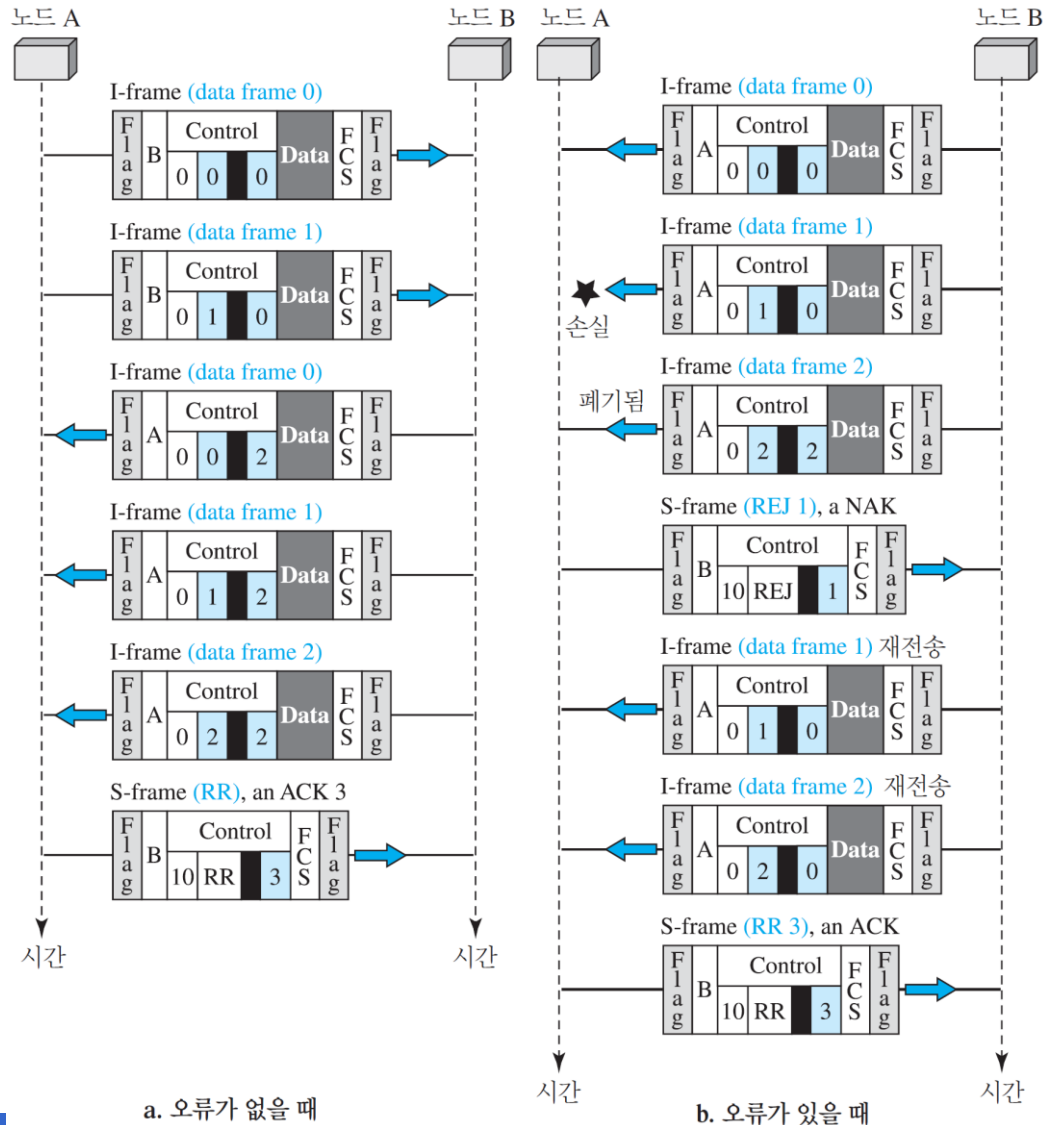
▶ 예제 11.18

- U-프레임을 사용하여 연결 설정과 해제를 하는지를 보여준다.



▶ 예제 11.19

- 피기배킹을 사용하여 두 가지 교환을 보여준다.



점-대-점 프로토콜

■ 점-대-점 프로토콜

- ▶ 점-대-점 프로토콜(PPP, Point-to-Point Protocol)
 - 점-대-점 접속을 위한 가장 널리 사용되는 프로토콜
 - 데이터 전송을 제어하고 관리하기 위해서 데이터 링크층에서 필요
 - 바이트 중심 프로토콜

점-대-점 프로토콜

■ 서비스

▶ PPP가 제공하는 서비스

- 장치들 사이에서 교환될 프레임의 형식을 정의
- 두 장치가 어떻게 링크 설정을 교섭하고 데이터를 교환할지를 정의
- 여러 네트워크층으로부터 페이로드를 받도록 설계
- 인증 또한 프로토콜에 선택적으로 제공
- 다중링크 PPP(Multilink PPP)은 다중 링크로 연결을 제공
- 네트워크 주소 구성을 지원
 - ✓ 가정 사용자가 인터넷에 연결하기 위해 임시로 네트워크 주소가 필요할 때 특히 유용

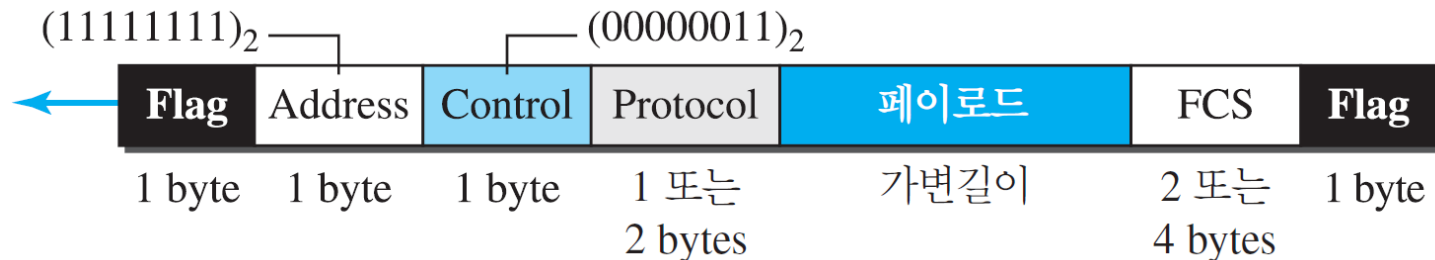
점-대-점 프로토콜

- ▶ PPP가 제공하지 않는 서비스
 - 흐름 제어를 제공하지 않음
 - 송신자는 수신자를 생각하지 않고 프레임을 계속 전송
 - 매우 간단한 오류 제어
 - ✓ CRC 필드가 오류 검출에 사용
 - 오류 제어를 하지 못하고 순서 번호가 없기 때문에 순서가 바뀌어 패킷이 도달할 수 있음
 - 다중점 링크에서 프레임을 다룰 수 있는 주소지정 방법이 없음

■ 프레임 짜기

▶ PPP 프레임의 형식

- 플래그: '01111110'
- 주소: 11111111(브로드캐스트 주소)로 설정
- 제어: 00000011로 설정, 흐름 제어도 제공하지 않음. 오류 제어 또한 오류 검출을 위해 제한됨
- 프로토콜: 데이터 필드에 사용자 데이터 또는 다른 정보가 들어 있는 것을 정의
- 페이로드 필드: 사용자 데이터나 그 밖의 정보들을 전달, 최대 1,500바이트
- FCS: 프레임 검사 순서 값(frame check sequence)

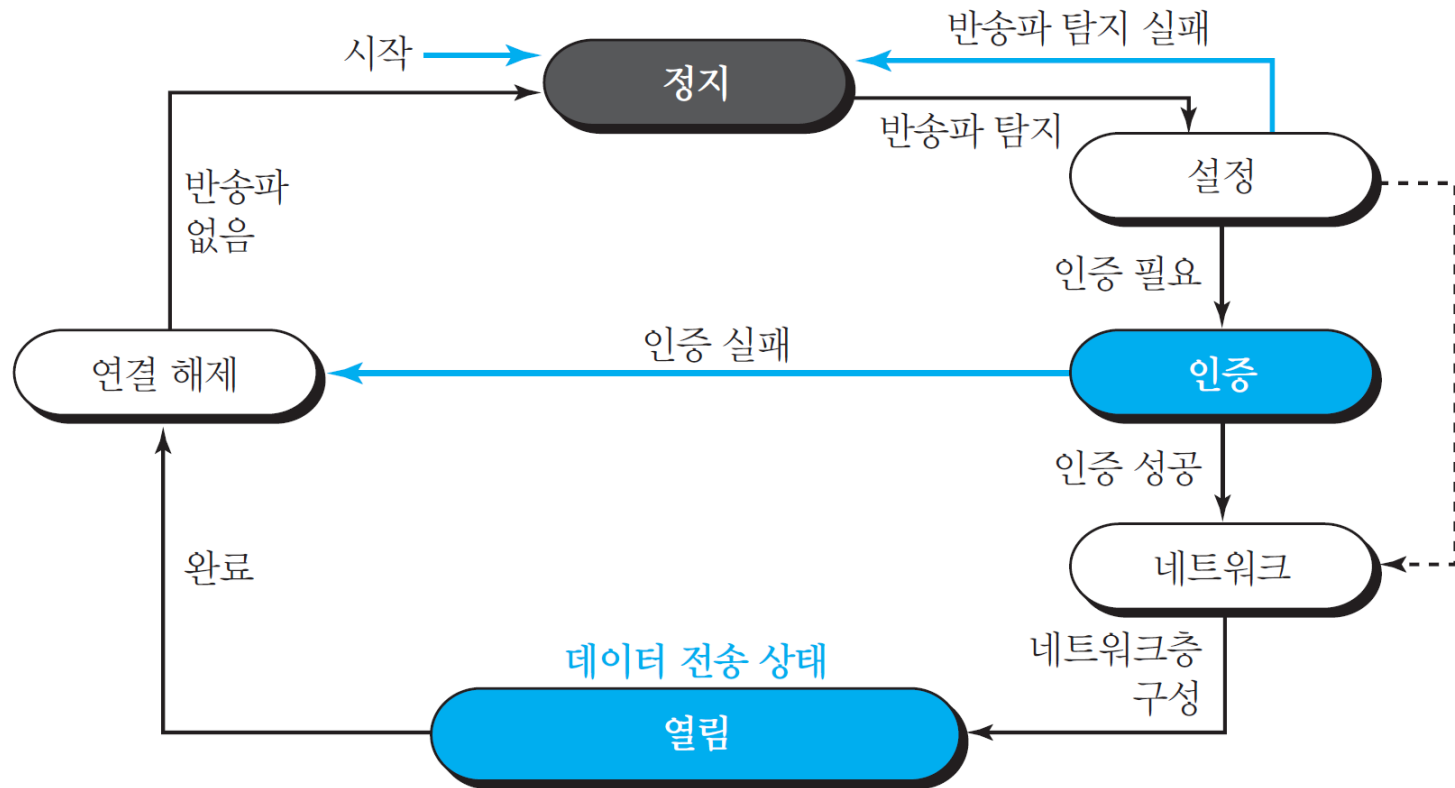


점-대-점 프로토콜

■ 천 이상 상태

- ▶ 정지(dead) 상태
- ▶ 설정(establish) 상태
 - 연결 확립
- ▶ 인증(authentication)
- ▶ 네트워크
 - 네트워크층 구성(프로토콜 선택)
- ▶ 열린(open) 상태
 - 데이터 전송
- ▶ 해제(terminate) 상태

점-대-점 프로토콜



점-대-점 프로토콜

■ 다중화

- ▶ PPP는 3종류 프로토콜을 사용
 - 링크 제어 프로토콜(LCP, Link Control Protocol)
 - ✓ 링크의 설정, 유지, 구성과 해제를 담당
 - 인증 프로토콜(AP, Authentication Protocol)
 - ✓ 당사자를 인증하기 위해
 - ✓ 2개의 AP
 - 네트워크 제어 프로토콜(NCP, Network Control Protocol)
 - ✓ 네트워크층의 데이터를 전송하기 위해
 - ✓ 여러 개의 NCP

점-대-점 프로토콜

범례

LCP: Link control protocol
AP: Authentication protocol
NCP: Network control protocol

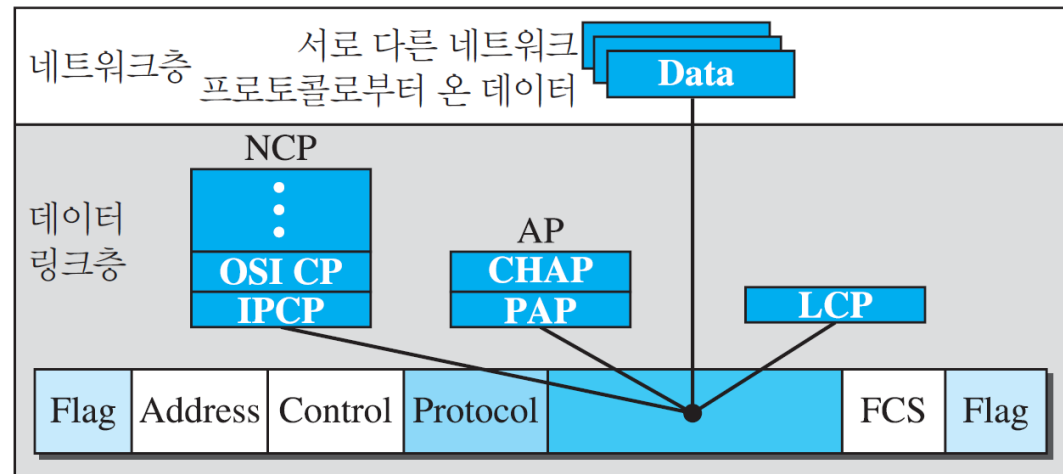
프로토콜 값:

LCP: 0xC021

AP: 0xC023 그리고 0xC223

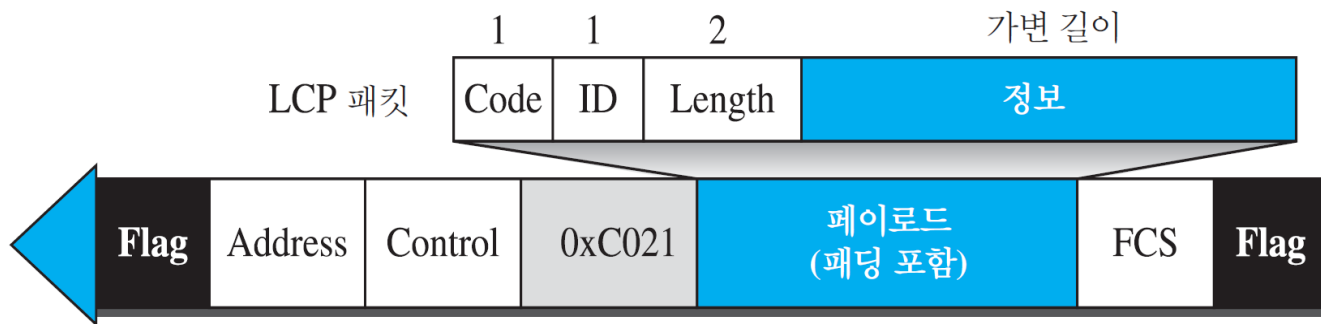
NCP: 0x8021 그리고

Data: 0x0021 그리고



점-대-점 프로토콜

- ▶ 링크 제어 프로토콜
 - 링크의 설정, 유지, 구성과 해제를 담당
 - 두 종단점 간의 선택사항을 결정하기 위한 협상 기능 제공



점-대-점 프로토콜

표 11.1 LCP 패킷

코드	패킷 유형	설명	
0x01	Configure-request	제안된 옵션과 해당 값의 목록이 들어있다	] 설정단계에 링크 구성을 위해 사용
0x02	Configure-ack	제안된 모드 옵션 수용	
0x03	Configure-nak	일부 옵션은 수용할 수 없음을 알린다	
0x04	Configure-reject	일부 옵션은 인식할 수 없음을 알린다	
0x05	Terminate-request	회선 종료 요청	] 해제단계에 연결해제를 위해 사용
0x06	Terminate-ack	종료 요청 수용	
0x07	Code-reject	알려지지 않은 코드임을 알린다	] 링크감시와 디버깅을 위해 사용
0x08	Protocol-reject	알려지지 않은 프로토콜을 알린다	
0x09	Echo-request	다른 종단이 사용 중인지 확인하기 위한 헬로 메시지 유형	
0x0A	Echo-reply	echo-request 메시지에 대한 응답	
0x0B	Discard-request	패킷 폐기 요청	

점-대-점 프로토콜

표 11.2 일반적인 선택사항

선택사항	기본값
Maximum receive unit (페이로드 필드 길이)	1500
Authentication protocol	없음
Protocol field compression	Off
Address and control field compression	Off

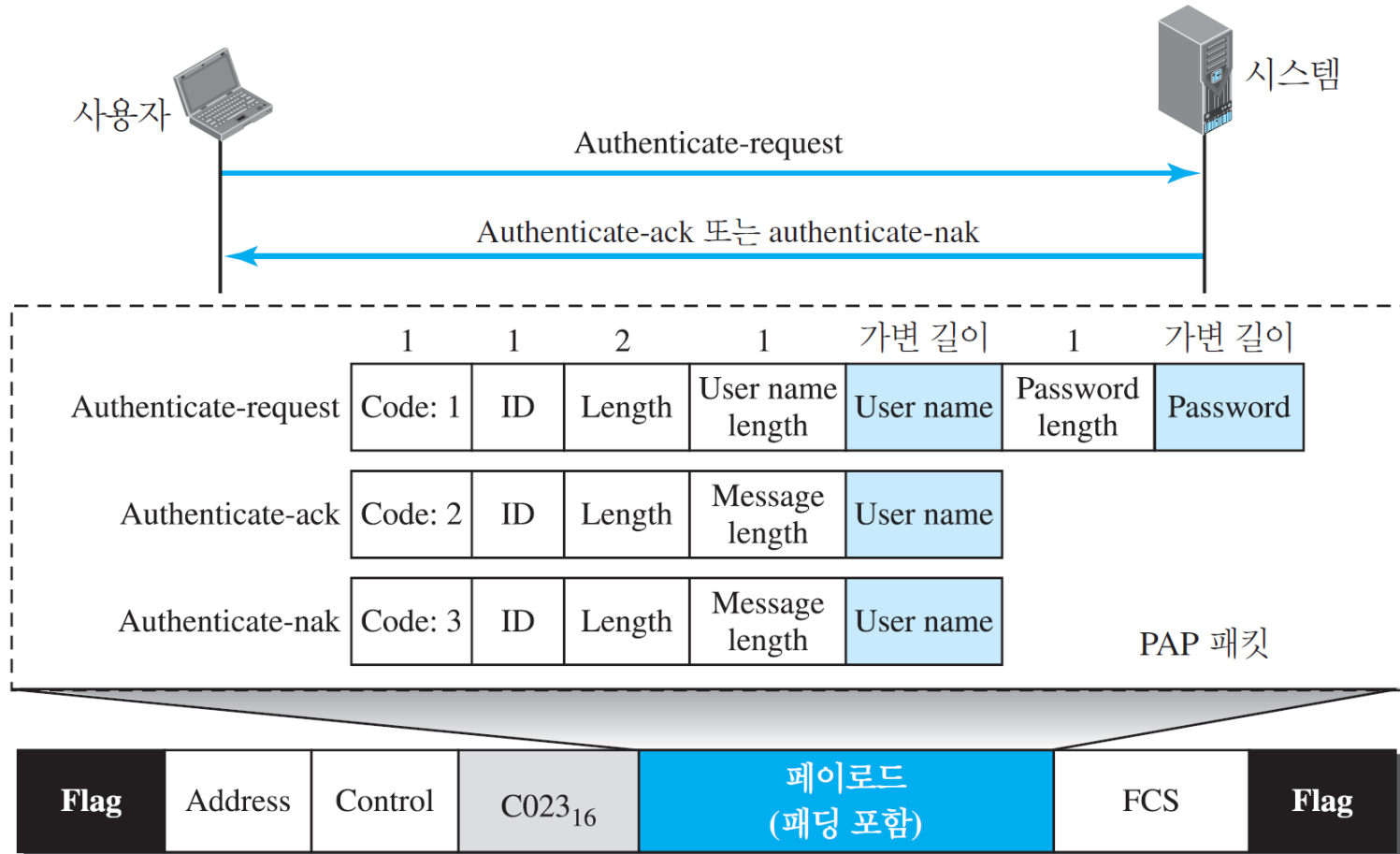
점-대-점 프로토콜

▶ 인증 프로토콜

- 사용자의 신원을 증명하기 위해 사용
 - ✓ PPP가 사용자의 인증이 필수적인 전화선 링크를 사용하도록 설계되었기 때문에
- 패스워드 인증 프로토콜(PAP, Password Authentication Protocol)
- 챌린지 핸드셰이크 인증 프로토콜(CHAP, Challenge Handshake Authentication Protocol)

- 패스워드 인증 프로토콜(PAP)

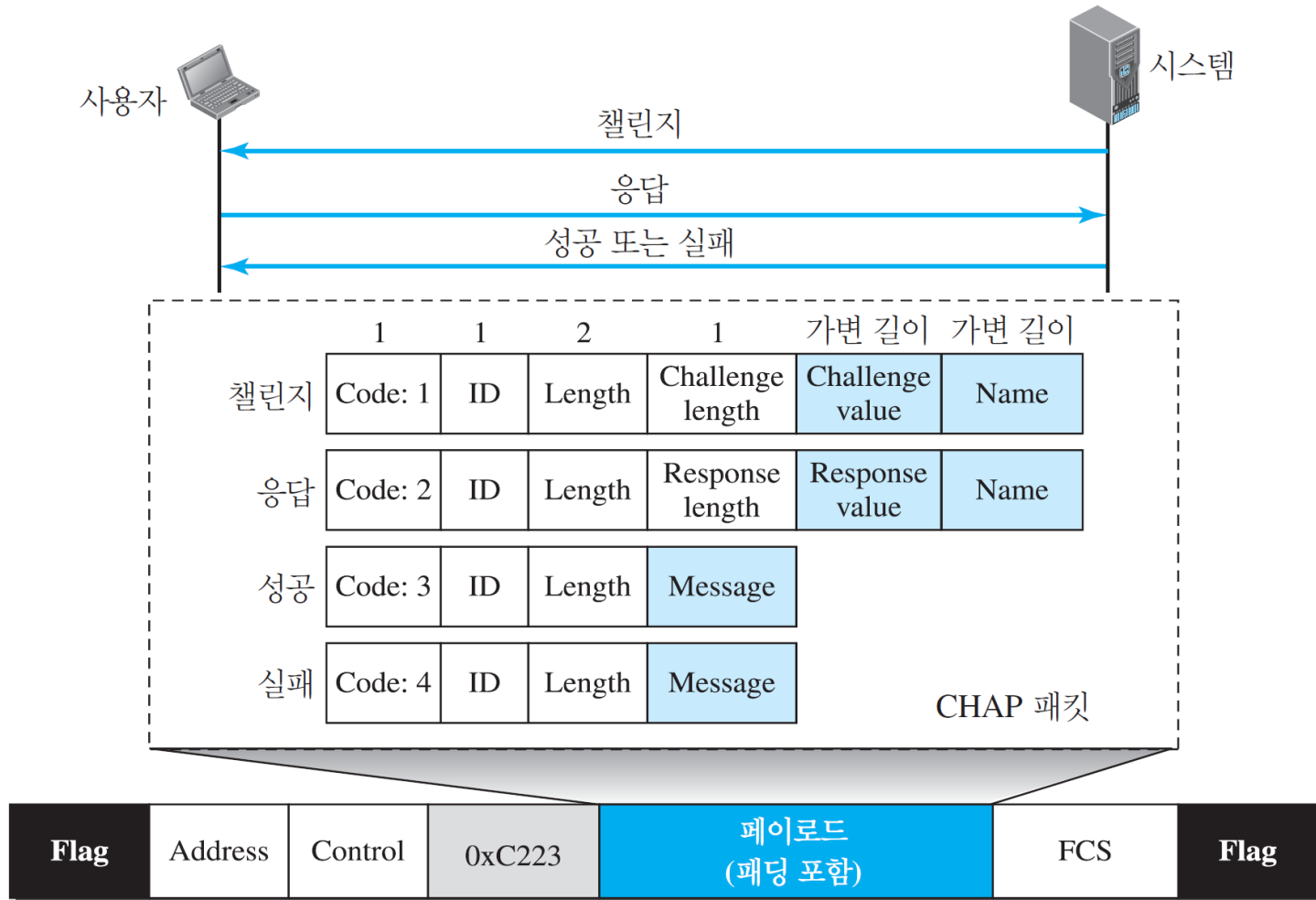
- ✓ 사용자는 인증 식별자(사용자 이름)와 패스워드를 전송.
- ✓ 시스템은 식별자와 패스워드의 유효성을 검사하여 연결을 설정하거나 거부



점-대-점 프로토콜

- 챌린지 핸드셰이크 인증 프로토콜
 - ✓ PAP보다 높은 보안을 제공하는 3-way handshake 인증 프로토콜
 - ✓ 시스템이 챌린지 값이 들어 있는 챌린지 패킷을 사용자에게 전송
 - ✓ 사용자는 챌린지 값과 사용자의 패스워드를 미리 정의된 함수를 적용하여 결과를 생성, 이 결과를 응답 패킷에 넣고 시스템에게 전송
 - ✓ 시스템도 사용자의 패스워드(시스템에게 이미 알려진 것)와 챌린지 값을 같은 함수에 적용하여 결과값을 생성
 - ✓ 만약 만들어진 결과가 응답 패킷으로 보내진 것과 같으면 접근이 허용되고 아니면 거부

점-대-점 프로토콜



점-대-점 프로토콜

- ▶ 네트워크 제어 프로토콜
 - PPP는 다중 네트워크층 프로토콜
 - ✓ OSI, Xerox, DECnet AppleTalk, Novel
 - 각 네트워크 프로토콜에 대해 특정 NCP를 정의
 - 인터넷네트워크 프로토콜 제어 프로토콜(IPCP, Internetwork Protocol Control Protocol)
 - 기타 프로토콜
 - ✓ OSI 네트워크층 제어 프로토콜
 - ✓ Xerox NS IDP 제어 프로토콜

- 인터넷워크 프로토콜 제어 프로토콜
 - ✓ IP 패킷을 위한 네트워크층 연결을 설정 또는 종료하는 패킷의 집합

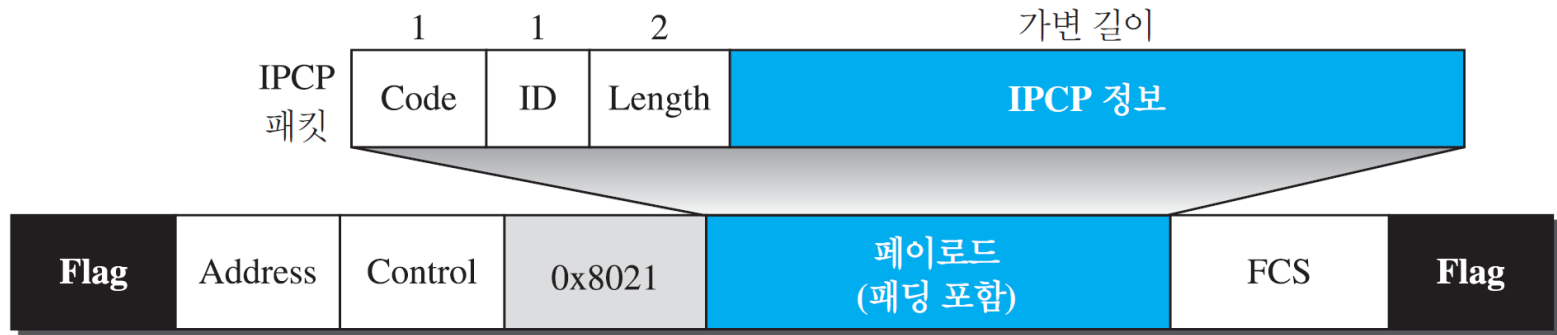
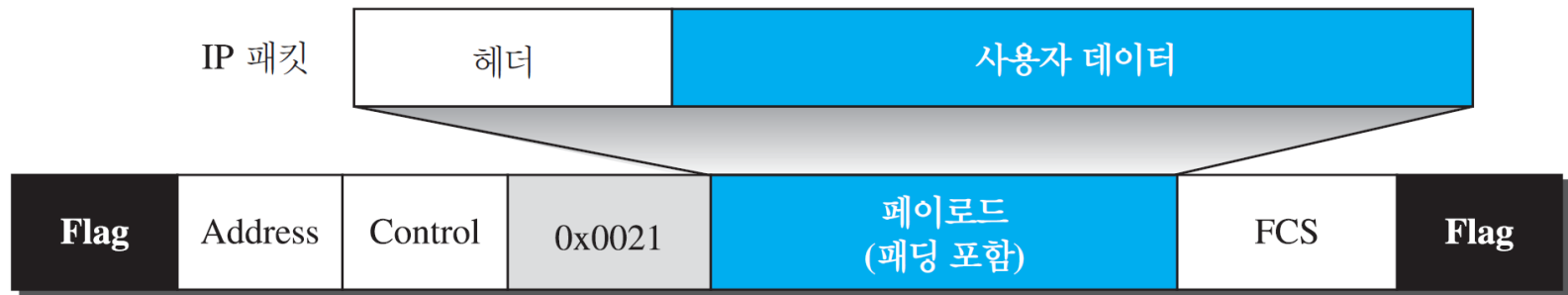


표 11.3 IPCP 패킷의 코드 값

코드	IPCP 패킷
0x01	Configure-request
0x02	Configure-ack
0x03	Configure-nak
0x04	Configure-reject
0x05	Terminate-request
0x06	Terminate-ack
0x07	Code-reject

점-대-점 프로토콜

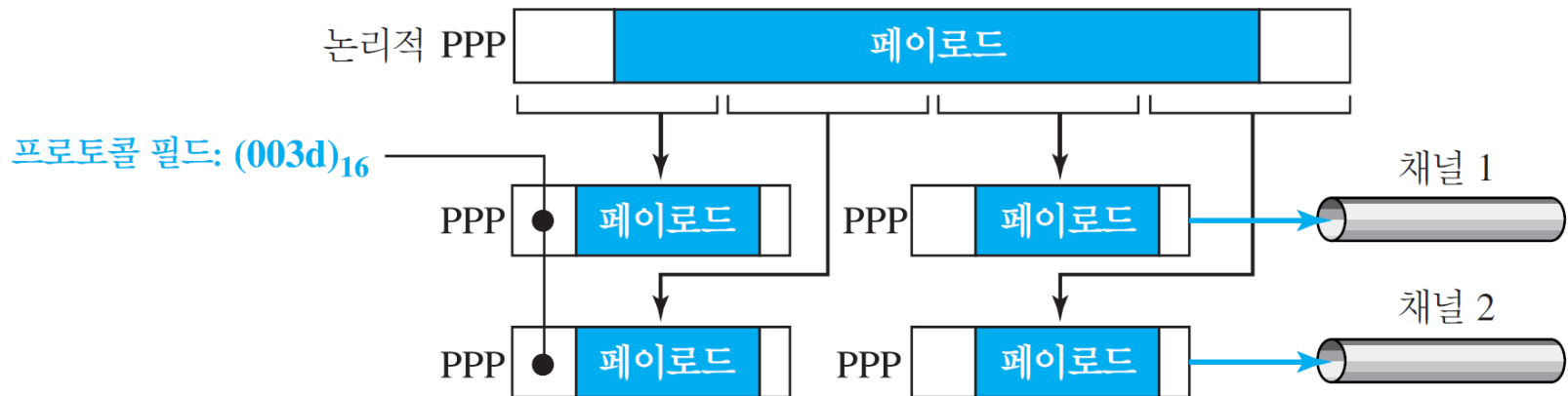
- ▶ 네트워크층으로부터 데이터



점-대-점 프로토콜

▶ 다중링크 PPP

- PPP는 단일 채널 점-대-점 물리 링크를 위해 설계
- 단일 점-대-점 회선에서 다중 채널이 가능해짐에 따라 다중회선 PPP가 개발



점-대-점 프로토콜

- ▶ 예제 11.10
 - 네트워크 패킷 전달의 천이 단계를 보여주고 있다.

